# LDAPX V2.2.0

# INSTALLATION GUIDE

*For z/OS*

**August 2022**

# Maintenance Level LD220E

**Subproducts and Fix Levels**

| Product | Load Fix | Panel fix |
|---|---|---|
| LDAPX V2.2.0 | - | - |
| | | |

# LDAPX maintenance level LD220E

## Program Information

This documentation refers to LDAPX from One Identity by Quest, version 2.2, modification level 0, maintenance level E, being the 5th release of this product. In this document and elsewhere this will often be abbreviated to just **LDAPX**.

LDAPX runs as a plug-in exit for 'IBM Tivoli Directory Server for z/OS' (also referred to as **TDS** in this document). When supplied as a component of One Identity Manager, LDAPX is also referred to as the 'RACF TDS Exit'.

The IBM TDS task runs as an LDAP server and, among other potential uses, it facilitates access to the RACF database by LDAP clients. TDS uses a program called **SDBM** to interface to RACF.

LDAPX **enhances** the functionality provided by SDBM by:

a) Overcoming the limit of 4096 results when listing USERs or GROUPs that is the default action in SDBM due to its use of the r_admin callable service.

b) Overcoming the limit of 4096 results when listing other RACF resource classes that is the default action in SDBM due to its use of the r_admin callable service.

c) Adding the ability to manage DATASET profiles and permissions.

d) Adding the ability to issue arbitrary TSO commands, clist calls, or rexx calls via an LDAP search, with output returned in attributes of the requested 'entry'.

Note that a valid license is required to use features b)  c) and d)

**Highest fixes included in Maintenance Level E**

The program build number is v64, this shows as startup message:
LDAPX Entered for Initialization - LDAPX 2.2.0 level E build v64

All updates previously issued for 2.1.0 as fix LDF2105 are incorporated in this release. See Appendix A for a record of features, fixes, and enhancements in each release of this product.

# Contents

**Program Information** ..... **2**

**Product installation** ..... **4**
Initial Phase of Installation – upload of data to z/OS ..... 4
Second Phase of Installation – Customisation using the @README Job ..... 4
Completing Initial Installation ..... 8
1) RACF updates ..... 8
2) SCHEMA update ..... 8
Installation Verification ..... 10
1) Starting TDS Successfully ..... 10
2) Running LDAPTEST Job ..... 10

**Installation Checklist** ..... **12**

**Troubleshooting Aid** ..... **15**

**Using the LDAPX Extended Features** ..... **18**
Syntax of LDAP commands recognised by LDAPX ..... 19

**Further Tailoring Considerations and Usage Notes** ..... **21**
LDAPX Configuration parameters ..... 21
The TDS DEBUG Trace ..... 21
LDAPX Execution Threads ..... 22

**LDAPXCFG Configuration File** ..... **24**
Introduction ..... 24
Syntax ..... 24
Keyword options ..... 24

**RACF permission considerations if TDS stc or bind user does not have SYSTEM SPECIAL** ..... **35**
General considerations ..... 35
SDBM backend ..... 35
Other resources ..... 36
Permissions for LDAPX ..... 36

**Appendix A – LDAPX Release History** ..... **39**
**Appendix B – LDAPX SMP/E Installation** ..... **42**
**Appendix C – LDAP Schema Updates** ..... **43**
**Appendix D - Using SSL/TLS to secure connections – possible scenarios** ..... **44**
**Appendix E – the @README install job** ..... **52**

# Product installation

This Installation Guide refers to One Identity LDAPX (extensions for IBM Tivoli Directory Server for z/OS), version 2.2, modification level 0, maintenance level E, being the 5th release of this product.

The product is supplied in one 936kb zip file which contains the following files:

- README.TXT (4kb)
- InstallE.pdf (770kb)
- LD220E.BIN (956kb)

InstallE.doc is the document you are reading.

This document covers the configuring of those parts of TDS necessary to use LDAPX, but for further information it is recommended to refer to the IBM TDS manual: 'IBM Tivoli Directory Server Administration and Use for z/OS'.

## Initial Phase of Installation – upload of data to z/OS

The initial phase of the installation consists of uploading the product code contained in the LD220E.BIN file to a z/OS system. The README.TXT file contains instructions on how to do that, but in summary it comprises:

- A binary file transfer of file LD220E.BIN to a sequential ('xxxx.vvvv.BIN') z/OS dataset of lrecl=80 blksize=3120.
- Using TSO command RECEIVE to create a PDS ('xxxx.vvvv.BINPDS') library from the BIN dataset.

The same dataset name high-level qualifier(s) chosen for the .BINPDS library (represented as 'xxxx.vvvv' above) will be used to create more datasets in the next phase of installation, so you may wish to chose a name unique to this particular version.

The .BINPDS library will contain several members, one of which is called @README. This member is used for the next phase of the installation as described below.

## Second Phase of Installation – Customisation using the @README Job

The @README member contains JCL which tailors various datasets/members to set up the environment to use LDAPX in the customer setting. There are numerous comments in the JCL which describe what changes are expected/necessary in order to customise the environment in the way appropriate to each customer setting. For reference the @README job is reproduced in the Appendix of this document.

The notes here concerning the running of the @README job should be considered **in addition to** reading the comments and instructions in the job itself.

Although contained in one member, the JCL is designed to be run as 2 or 3 separate jobs, with the 'RESTART=' parameter on the jobcard utilised to start the job at the desired point. For simplicity, the restart labels of PART1, PART2, and PART3 have been defined. Note that the 'end of job' marker '//  ' will need to be changed in some places to e.g. a '//*  ' (comment card) to allow the job to run correctly. The comments in the JCL will explain when this is necessary. Please be aware that such changes may need to be reversed if any steps need to be rerun.

The PART1 processing run does the final 'unpacking' of the members in the BINPDS datasets to create the full set of datasets used by the LDAPX product. PART2 customises some of the supplied material using values chosen by customer. PART3 copies tailored parts to the places needed for running the TDS task.

Which parts to run will depend on whether this is a completely new installation of LDAPX or whether the customer alreadys runs/uses an earlier release of LDAPX and/or the the Tivoli Directory Server (TDS) for z/OS task. Even if a previous release of LDAPX is in use, there can be some merit to carrying out the installation as if it were completely new because it makes clear any changes introduced with this release. This can be done simply by choosing appropriate values for variables so that a distinct set of output datasets and directories are updated by the @README jobs.

The first thing to change in the @README job is the jobname and other parameters on the jobcard such as CLASS so that the job conforms to any in-house standards and will run successfully.

The following summarises what each part of the @README job does and what information you might need to supply.

**PART1**

The steps in PART1 use the following:

- the 'xxxx.vvvv.BINPDS' dataset from initial installation step and
- the values of the JCL variables 'PREFIX', 'PRESMP', and 'DISKVOL'.

**There is an assumption that the value of PREFIX will be the same as that chosen for the .BINPDS dataset, i.e. @README PART1 assumes that dataset &PREFIX.BINPDS was created in initial installation step.**

Set value of DISKVOL to the volume serial of an appropriate DASD volume where new datasets for this product can be allocated. (It is possible that local SMS rules may override what is chosen).

PRESMP only needs to be used if an SMP install is required. See Appendix B for information about setting PRESMP variable if you wish to do this.

After a successful execution of part 1 the following datasets will exist and some will contain members ready for use in later steps:

&PREFIX.TEMP
&PREFIX.INSTALL
&PREFIX.CNTL (*)

&PREFIX.LOAD (*)
&PREFIX.REPRO (*)
&PREFIX.FIXLIST (*)

**If you are doing an SMP install** the dataset names above marked (*) will be in the format &PREFIX.&PRESMP.CNTL, &PREFIX.&PRESMP.LOAD, etc.. There will also be a dataset called &PREFIX.&PRESMP.JCLIN. See Appendix B for details of the extra jobs that need to be run at this point for SMP install. Following a successful run of those jobs all the datasets as in the list above (and some others) are ready for running PART2.

**PART2**

The steps in PART2 require:

- some of the datasets/members created/populated by PART1
- the values of the JCL variables 'PREFIX' 'USSDIR' 'STEPLIB' 'CLIB' 'SYSEXEC' 'ISPLLIB' 'SYSUADS'
- IBM supplied TDS installation files

The installation of LDAPX requires changes to the IBM-supplied TDS configuration file (ds.conf), the environment variables (ds.envvars), and the TDS started task PROC (DSSRV). The steps in PART2 copy those supplied components into datasets used for this installation and changes them there so they are kept separate from existing system (for now). This relies on the following assumptions:

a) IBM Supplied ds.conf and ds.envvars are in z/Os Unix directory **/usr/lpp/ldap/etc**
b) IBM Supplied DSSRV is in dataset **GLD.SGLDSAMP**
c) The above files are as originally supplied (though the process should still work as long as they have not been changed too much)

If either of those TDS installation datasets are NOT in the locations shown above then before submitting PART2 the 'COPYDS' step in @README will need to be changed to specify the correct one(s):

```
//COPYDS  EXEC PGM=IKJEFT01
//DSIN1     DD PATH='/usr/lpp/ldap/etc/ds.conf',
//             PATHOPTS=(ORDONLY)
//DSIN2     DD PATH='/usr/lpp/ldap/etc/ds.envvars',
//             PATHOPTS=(ORDONLY)
//DSIN3     DD DSN=GLD.SGLDSAMP(DSSRV),DISP=SHR
```

The value of USSDIR needs to be set to the name of a z/OS Unix directory that will be used for this installation of LDAPX. Note that although the directory *name* is used in PART2 the directory itself is not actually used until PART3 is run.

The value of STEPLIB is the name of a load library into which the LDAPX load module is copied and it will be used as the STEPLIB of the tailored DSSRV proc. Note that although the dataset *name* is used in PART2 the dataset itself is not actually used until PART3 is run.

The values of JCL variables CLIB, SYSEXEC, ISPLLIB and SYSUADS can be set to the names of datasets that will be allocated to the invoking task if/when LDAPX is used to issue TSO commands or manage datasets.

The values entered here are used to automatically tailor both the LDAPX configuration file and the SCRIPTSO command. As these dataset names can be changed by updating the configuration file manually at a later date the **recommendation is to NOT change the values** in the @README job.

After a successful run of PART2 there will be:

- Tailored copy of LDAPX component SCRIPTSO in &PREFIX..REPRO(SCRIPTSO)
- Tailored copy of initial LDAPX configuration file in &PREFIX..CNTL(LDAPXCFG)
- Tailored copies of supplied IBM TDS components ds.conf, ds.envvars and DSSRV in &PREFIX..CNTL(DSCONF) , &PREFIX..CNTL(ENVARS), &PREFIX..CNTL(DSSRV) respectively
- An installation script created/tailored for use in PART3 in &PREFIX..TEMP(TSOOMVS)

**PART3**

**The steps in PART3 copy newly tailored files back to the datasets/directories specified in earlier steps. If any of those datasets or directories are currently in use by TDS and/or LDAPX you should exercise caution before running PART3 without modification to ensure you do not overwrite anything useful.**

The steps in PART3 require:

- The tailored script in &PREFIX..TEMP(TSOOMVS)
- the values of the JCL variables 'PREFIX' 'USSDIR' 'STEPLIB' 'PROCLIB'
- that the z/OS Unix directory specified in JCL variable USSDIR exists
- that the dataset specified in JCL variable STEPLIB exists
- knowledge of what run-time directory is used for LDAP (TDS) configuration file (ds.conf) if any
- The tailored copies of IBM TDS components ds.conf, ds.envvars and DSSRV in &PREFIX..CNTL(DSCONF) , &PREFIX..CNTL(ENVARS), &PREFIX..CNTL(DSSRV) respectively

**The installation process assumes that /etc/ldap** is the z/OS Unix directory used for running the TDS proc, i.e. the location of the ds.conf and ds.envvars files. If PART3 is run as supplied the newly tailored copies of ds.conf and ds.envvars will be copied to that directory, **overwriting the current files of that name** (though it does make a copy of them first). You should examine/change the 'LDINST' step of the @README job if that is not the correct directory.

The z/OS Unix directory specified in JCL variable USSDIR is written to in PART3 so it **must exist** before PART3 is run. The directory must be capable of containing executable scripts (e.g. by issuing chmod 775 *dirname)* .

The newly-tailored DSSRV proc is copied to the dataset specified in JCL variable PROCLIB in PART3. The default action is to NOT overwrite a member of the same name if it already exists in that dataset. The 'PROCLIB' step of the @README job should be examined and changed to carry out action required. It is assumed this dataset is one which contains system started-task procs, if it does not, the DSSRV proc must be copied to such a dataset subsequently.

The load library specified in JCL variable STEPLIB has the LDAPX load module copied to it in PART3. The default action is to NOT overwrite a module of the same name if it already exists in that library. The 'STEPLIB' step of the @README job should be examined and changed to carry out action required.

## Completing Initial Installation

### 1) RACF updates

Review RACF considerations in this document and in the job supplied in member LDAPRACF of the &PREFIX.CNTL library, and issue RACF commands required for running IBM TDS for z/OS with LDAPX. For a new install of TDS on a test system a simple RACF regime for it can be set up using LDAPRACF as supplied (other than the jobcard). Some of the steps that follow such as update of schema and initial verification use the supplied RACF definitions by default. If you are already running IBM TDS you may have already performed many of the commands so may need to adapt the list in LDAPRACF to fit your existing RACF settings.

Ensure that the userid running the jobs has enough authority for the RACF commands to execute successfully.

### 2) SCHEMA update

To support the extra features provided by the LDAPX TDS plug-in and run in 'full extensions' mode it is necessary to update the default TDS LDAP schema to add new elements supplied with LDAPX. If you have no plans to use the extra LDAPX features then this step could be omitted.

To update the LDAP schema, the TDS task needs to be running. If you have not used TDS previously it is quite likely at this point that some further tailoring will be needed to get the TDS and LDAPX configurations set up correctly for your requirements. However, the tailoring already completed should be sufficient to allow the TDS started task to be brought up.

To start TDS issue MVS command 'S DSSRV' (or whatever name the TDS started task proc has been given). If you experience problems starting the TDS task then refer to the Installation Checklist section of this document for ideas of what might need changing.

Review and submit the job supplied in member LDAPMOD of the &PREFIX.CNTL library. This runs the IBM utility ***ldapmodify*** which converts a supplied file of LDIF statements into LDAP requests to add extra LDAP attributes to the schema in order to support the extra features provided by the LDAPX plugin. This job can be rerun without harm. It can also, if edited, be used to revert to the standard TDS schema (it contains some commented-out lines to delete file /var/ldap/schema/schema.db which TDS will recreate when next started).

If the LDAPX installation process has been used with default installation options this job should run with no problems. The ldapmodify command specifies userid LDAPADM and the password of that id as set by the default installation process. More specifically the ldapmodify command specifies the userid by using the 'distinguished name' (DN) and for success that has to match the AdminDn in the TDS configuration file (ds.conf).

Note that this job does create a lot of output and it should be examined carefully to check if the schema update has actually happened correctly. The following lines denote a successful update

```
modifying entry cn=schema
modify complete
```

If the schema update has not been successful then refer to the Installation Checklist section of this document for ideas of what might need changing.

Appendix C lists all the new object and attribute elements added to the default schema for LDAPX use.

## Installation Verification

A job is supplied in member LDAPTEST of the &PREFIX.CNTL library which can be used to verify that the TDS and LDAPX configuration is working. The TDS started task (DSSRV) must be running for the LDAPTEST job to run successfully.

### 1) Starting TDS Successfully

To start TDS issue MVS command 'S DSSRV' (or whatever name the TDS started task proc has been given). As the TDS started task comes up it initialises LDAPX (the SDBM2TSO module) and the following message will be displayed:

```
LDAPX Entered for Initialization - LDAPX 2.2.0 level E build v64
```

If you do not see that message, check that the SDBM2TSO load module and load library are set up correctly and accessible to the TDS started task proc as described in the Installation Checklist section of this document. Also check that the TDS configuration file (ds.conf) has the 'plugin preOperation SDBM2TSO…. ' and 'plugin postOperation SDBM2TSO… ' and other relevant statements as described in the Installation Checklist.

If you do see the above message you should then see one of the following messages too:

**LDAPX running in full extensions mode on cpuid *yourCPUid***

**or,**

**LDAPX running in basic mode on cpuid *yourCPUid* to support more than 4096 users & groups**

LDAPX runs in basic mode if it does not find a valid license code for this CPU in the LDAPXCFG configuration file. If you are expecting LDAPX to be in full extensions mode it could be that the LDAPXCFG configuration file is not specified correctly in the TDS started task proc, or that the **LicenseCode** statement in LDAPXCFG is missing, or the **LicenseCode** statement contains a value not valid for the CPU on which it is running. Again refer to the Installation Checklist.

If you wish to use the facilities provided by LDAPX in full extensions mode then contact Quest / One Identity support to obtain a valid license code.

The LDAPTEST job described below runs a test which requires LDAPX to be in full extensions mode to complete successfully.

### 2) Running LDAPTEST Job

The LDAPTEST job creates and runs a script which uses the ***ldapsearch*** command to send queries to the TDS LDAP server, with the output from the commands copied back to the spool so the command results can be seen in the job output. The ldapsearch commands entered cause TDS to invoke the LDAPX plug-in exit.

Firstly make changes to the jobcard to ensure it is valid for your environment. Then change all instances of the string ‘HLQ’ to a valid userid or other high-level qualifier that the job submitter can create. (There are 4 instances including one in a comment).

The job sets and uses the values of z/OS unix variables to build the necessary ldapsearch command strings. The variables appear after the //SERVER DD card with the initial settings:

```
host="127.0.0.1"
port="389"
suffix="cn=racf,o=site"
binduser="ldapadm"
bindpw="notthis"
```

The values are those defaulted/suggested during the installation tailoring. If different values have been chosen during installation, the values of the z/OS unix variables in the job should be changed accordingly.

If problems are still encountered when running the job then refer to the Installation Checklist section of this document for ideas of what might need changing.

If run as supplied the first ldapsearch command looks for all RACF user entries starting with ‘A’. If there are some such user profiles in your RACF dataset they will be listed. The second ldapsearch command shows an example of using the TSOCMD feature to issue a TSO command and have the results returned. If this runs successfully it proves that a valid license has been entered because that LDAPX feature does not work without a valid license.

# Installation Checklist

This section and the next provide information to help you work out the potential cause of problems if things are not working as you would expect with your TDS / LDAPX installation.

The table in this section is probably of most use if problems have been encountered when bringing up the TDS/LDAP task when trying to do the schema update or installation verification, or even before you attempt to do those. Review the list of components that comprise the TDS / LDAPX solution and ensure the tailoring on your system matches what tailoring is expected.

The next section lists common problems/situations you may encounter and suggests possible causes and/or remedies.

| **Component** | **Notes / Expected Tailoring** |
|---|---|
| **TDS started task** | Usually called DSSRV (installation process assumes this).<br>JCL for DSSRV needs to be in a system proclib library.<br>RACF id assigned to started task in installation process is LDAPSRV.<br>Requires a STEPLIB for the load library containing LDAPX SDBM2TSO module (unless that module is placed in a LINKLIST library).<br>Has a DD card LDAPXCFG to allocate the LDAPX configuration file.<br>Must use the 31 bit version of the GLDSRVxx module, i.e. `EXEC PGM=GLDSRV31` .<br>Opens/uses the TDS configuration file (ds.conf) in the ldap runtime directory e.g. using the PARM '-f' on the EXEC statement<br>`//GO EXEC PGM=GLDSRV31,REGION=&REGSIZE,TIME=1440,`<br>`// PARM=('/&PARMS -f /etc/ldap/ds.conf >DD:DSOUT 2>&1')` |
| **TDS configuration file** | Usually called ds.conf in /etc/ldap directory (the LDAP run-time directory).<br>Must contain the following statements to enable LDAPX use:<br>plugin preOperation SDBM2TSO plugin_init "*logdir*"<br>plugin postOperation SDBM2TSO plugin_init "*logdir*"<br>database SDBM GLDBSD31/GLDBSD64<br>enableResources on<br>suffix "……"<br>Installation process sets *logdir* above to "/usr/oneiden/logSDBM2TSO" (depends on &USSDIR value chosen in @README job).<br>Installation process sets 'suffix' above to "cn=racf,o=site" .<br>Installation process sets 'AdminDN' to "racfid=ldapadm,profiletype=user,cn=racf,o=site" |
| **LDAPSRV userid** – (or equivalent) | Installation process defines this RACF id for use by the DSSRV started task.<br>Ideally defined as SYSTEM SPECIAL.<br>Has OMVS superuser attribute (uid 0).<br>Has TSO segment defined . |

| | |
|---|---|
| **LDAPADM userid** – (or equivalent) | Installation process defines this RACF id as the 'bind user' for LDAP client commands .<br>Userid used for ldapmodify and ldapsearch commands in installation jobs.<br>This userid (or a similar one) would be used by an LDAP client sending requests to this TDS |
| **SDBM2TSO module** | Resides in STEPLIB dataset of DSSRV started task OR dataset in LINKLIST.<br>Must be an APF authorised dataset.<br>If RACF program control active the dataset must have access to ** profile in PROGRAM class. |
| **LDAP Schema** | Needs modifications to enable use of LDAPX extra features.<br>Supplied installation job LDAPMOD updates schema.<br>Location of schema is a z/OS Unix file defined by TDS configuration file parameter 'schemapath' , default /var/ldap/schema/schema.db .<br>Userid specified on ldapmodify command in LDAPMOD job must have write access to z/OS Unix directory and schema.db file .<br>Userid specified on ldapmodify command in running LDAPMOD job must be the one specified in the TDS configuration file (ds.conf) AdminDN statement . |
| **LDAPX z/OS Unix installation directory** | As specified in USSDIR JCL variable in @README job, default value is /usr/oneiden .<br>LDAPSRV userid needs write access to that z/OS Unix directory. |
| **LDAPX configuration file** | The dataset allocated to DD name LDAPXCFG in the DSSRV proc.<br>LDAPX will run in basic mode if configuration file not accessible.<br>Installation process creates sample in &PREFIX.CNTL(LDAPXCFG) . |
| **Scriptso script** | Used if full use of LDAPX facilities is required (i.e. TSOCMD, dataset and special class RACF profile management).<br>Copied by default to the LDAPX z/OS Unix installation directory.<br>Location specified in LDAPX configuration file statement '**scriptsoPath**' .<br>Must be marked as executable. |
| **IP address and port** | The IP address and port that this TDS task listens on is defined in ds.conf.<br>The ldapmodify and ldapsearch commands used in the LDAPMOD and LDAPTEST jobs specify the host IP address as 0.0.0.0 or 127.0.0.1, which in most installations will equate to the host on which the job is running; if those jobs aren't working, try changing the IP address in those commands to the correct one for the host on which the TDS task is running.<br>Similarly, the supplied jobs assume port 389 is being used by TDS task, so they should be changed if the TDS task uses a different port.<br>If remote LDAP clients are to connect to TDS then company network measures may need to be considered to ensure the traffic can flow between them . |

| | |
|---|---|
| **RACF** | It is very likely RACF changes will be necessary in order to make TDS work with LDAPX. Please refer to other sections of this document and the comments in the &PREFIX.CNTL(LDAPRACF) job for more detailed guidance, but note especially:<br>If using 'useBinduser' you will need to define SURROGAT class profiles<br>If using 'FacilityPermit' you will need the FACILITY class profile 'ONEIDEN.LDAPX.PERMIT'<br>Ensure the userid(s) used to create/define datasets and other resources during installation have the necessary RACF authority to do it successfully. |

## Troubleshooting Aid

The following table contains symptoms you may encounter when trying to use TDS/LDAPX from One Identity Manager (or possibly an LDAP browser client).

**In all cases**, when encountering any of the symptoms below (or others), the best first action to take is to run the &PREFIX.CNTL(LDAPTEST) job. If that job runs successfully it immediately shows that the TDS / LDAPX set up is substantially correct and rules out some of the possible causes. If it doesn't run correctly then it may provide a different and/or clearer picture of where the problem might be. Also, there is merit in starting the TDS debug trace before re-trying the failing process (and/or the LDAPTEST job if that is failing) so that as much information as possible can be gathered to help find cause of the problem. (The TDS debug trace is mentioned elsewhere in this document).

| Symptom | Possible Cause | Notes / Things to Check/Do |
|---|---|---|
| | | |
| Messages at client end suggest 'no connection made' or 'failed to connect' | TDS task is not up or not responding. | Make sure DSSRV task is running.<br>Look for messages in DSSRV task output and/or z/OS system log that correspond with client message times. |
| | Client trying to connect on wrong IP address / port.<br>Firewall is stopping IP traffic between client and TDS server. | Check 'listen ldap' and/or 'listen ldaps' statements in the TDS config (ds.conf); when DSSRV task starts it lists all the interfaces it is listening on.<br>Check LDAP client configuration - is it using IP address / port consistent with above server configuration.<br>If possible try a request from a different client. For e.g. if LDAPTEST job runs successfully it implies problem is more likely to be connectivity of other client. |
| | Trying to make secure (ldaps) connection and SSL handshake failing. | If failure when trying to connect using SSL try connecting non-SSL.<br>SSL handshake failure could have many potential causes; need to check certificates at both client and server end, also cypher suites; might be useful to start an IP packet trace that can show why handshake fails. |
| (continued…) | | |

| | | |
|---|---|---|
| Error messages at client such as 'Invalid DN syntax' or 'No such object' | Transaction uses extended features of LDAPX and schema not updated to include required elements.<br><br>(If DEBUG trace active and 'MUTEX WAIT' messages are seen in TDS log) Transaction uses extended features of LDAPX and TDS still processing a previous transaction.<br><br>Transaction uses extended features of LDAPX and LDAPX module not being called by TDS.<br><br>Some element of DN in transaction not recognised by LDAP. | Check that the installation job LDAPMOD completed correctly (or just run it again).<br><br>See section in this document about 'LDAPX Execution Threads' for explanation / actions around this<br><br>Ensure 'plugin' statements are in TDS config file and that they reference SDBM2TSO (see checklist above).<br><br>Confirm value of 'suffix' parameter in TDS config (ds.conf) is correct; when TDS task (DSSRV) starts it lists all the configuration values in effect – check suffix value is as expected.<br>Check client end is aware of correct suffix to use when forming DN.<br>Check client end is aware of correct syntax to use when forming DN. |
| (continued…) | | |

| | | |
|---|---|---|
| No obvious errors, but not having expected outcome (i.e. no output returned at all, less profiles returned than expected, TSO command not issued, etc.) | RACF id used by DSSRV started task does not have sufficient authority | Refer to information in this document and in the LDAPRACF job and check all RACF authorisations are in place. RACF 'System Special' is recommended |
| | If using 'facilityPermit' and/or 'usebinduser', scope of RACF authority of binduser doesn't cover all profiles expected. | Check binduser RACF authority is as expected by issuing RACF commands in batch or TSO. |
| | If using 'facilityPermit' and/or 'usebinduser', there is no SURROGAT class profile defined for this binduser. | Refer to information in this document and in the LDAPRACF job and check all RACF authorisations are in place. |
| | If using 'FacilityPermit' the FACILITY class profile 'ONEIDEN.LDAPX.PERMIT' is missing/incorrect. | Refer to information in this document and in the LDAPRACF job and check all RACF authorisations are in place. |
| | RACF id being used as binduser is system special or has required RACF authority BUT there are binduser definitions in LDAPXCFG and this RACF id is not one of them | If there is at least one binduser defined in the LDAPXCFG file then all bindusers must be defined there, even if binduser id is system special |
| | DSSRV Racf id and/or binduser do not have OMVS segment defined | Refer to information in this document and in the LDAPRACF job and check all gid and id definitions are as expected. |
| | If using LDAPX extended facilities i.e. TSOCMD, dataset profile, or other class management, the **scriptso** script not accessible/executable by DSSRV started task id or binduser | Refer to information in this document and the @README job about setting up the LDAPX installation directory, the **scriptso** script, and the scriptsopath configuration statement |
| | If using LDAPX extended facilities i.e. TSOCMD, dataset profile, or other class management, **tsocmdxxxxxxx** configuration statements incorrect | Refer to information in this document and comments in the supplied LDAPXCFG member in the &PREFIX..CNTL dataset to set up correctly |
| | LDAPX not being called by DSSRV | Ensure all tailoring as shown in the 'Installation Checklist' above is in place |
| | Dataset profile name specified in lower case when using profiletype=DATASET | RACF dataset profile names use upper-case letters |
| | Requests using LDAPX extended facilities not being processed by LDAPX due to requests arriving faster than they can be dealt with | See section in this document about 'LDAPX Execution Threads' for explanation / actions around this |

## Using the LDAPX Extended Features

LDAPX is designed to be used by One Identity Manager to allow it to manage resources stored in a z/OS mainframe RACF database. Because of this, users of One Identity Manager use the facilities of that product to manage RACF users, so ordinarily there is no need to know the details of the LDAPX interface that One Identity Manager uses.

However, for the purposes of testing and when trying to isolate problems it is very useful to be able to invoke LDAPX from outside of One Identity Manager. This is done by using an LDAP client program. The easiest client to use for testing is the one supplied on z/OS but any LDAP client that runs on any device could be used, assuming that device has a network connection to the TDS for z/OS LDAP server. One such client is LDP.EXE but there are many others. Using the z/OS LDAP client is good to ensure the LDAP server is running properly and servicing requests without worrying about the potential connectivity problems a remote client might have.

The extended features provided by LDAPX are implemented by adding new object and attribute elements into the LDAP schema. This means that when a client sends an LDAP command which refers to these elements, LDAPX knows it needs to take action. LDAPX then interprets the 'Base Entry Distinguished Name' (BaseDN) string to discover the desired action. This is best illustrated with an example. With a command containing BaseDN:

Racfclassact=XXXXXX,profiletype=TSOCMD,cn=racf,o=site

The PROFILETYPE of TSOCMD is recognised by LDAPX as one of its 'special' objects and thus something it needs to handle. It then takes the value of RACFCLASSACT – XXXXXX in this example – and issues it as a TSO command. All the output from the command is captured and returned to the requestor as the value of attribute 'RACFprogrammername'.

Note that the remaining part of the command as shown above is the same as the suffix defined in the TDS configuration file (the default value). This indicates that the object in the query is to be handled by SDBM for this particular TDS task.

What isn't shown in the query above is the host and port number necessary to direct the query to the correct TDS server, or the credentials to be used when doing the query. That is done differently for different LDAP clients. For the z/OS TDS ldapsearch command all that information is in the command parameters. For a client like LDP.EXE the connection and bind are done up front and they maintain across queries until they are explicitly changed.

Appendix C lists all the new object and attribute elements added to the default schema for LDAPX use.

The next section has the syntax of the BaseDNs which invoke the extended features.

## Syntax of LDAP commands recognised by LDAPX

As in the example in the previous section, only the baseDN part of the command is shown as that is common to all clients.

| LDAP command and baseDN | What LDAPX does |
|---|---|
| Perform LDAP search for baseDN= **racfclassact**=*tttttt*,**profiletype=TSOCMD**,<suffix> | Issues command *tttttt* in a TSO environment |
| Perform LDAP add/update/delete/search for baseDN= **racfDataset**=*profilename*,**profiletype=DATASET**,<suffix> | Perform LDAP action on RACF dataset profile *profilename* |
| Perform LDAP add/update/delete/search for baseDN= **racfPermitId=***userid,***racfDataset**=*profilename*,**profiletype=DATASET**,<suffix> | Uses value of attribute **racfaccess** to change the access *userid* has to dataset profile *profilename* |

Note that the profiletype values TSOCMD and DATASET should be specified in upper-case. Also the profile name specified as the value of racfDataset should be in upper-case.

For example, doing a search for base entry DN…

**racfclassact=TIME,profiletype=TSOCMD**,cn=RACF,o=SITE

…would issue the TIME command on a TDS system where cn=RACF,o=SITE has been specified as the suffix parameter for the SDBM backend (in ds.conf). The output would be returned in (by default) values of attribute racfprogrammername in this form…

```
000 ***** TOP OF RESULTS *****
001 TIME-03:49:56 PM. CPU-00:00:00 SERVICE-2127 SESSION-00:00:01 MAY 23,2016
002 ***** END OF RESULTS *****
```

Using the z/OS Unix ldapsearch command to invoke the above would look like:

```
ldapsearch -s one -h 10.10.180.222 -p 389 -D "racfid=LDAPADM,profileType=User,cn=RACF,o=site"
  -w notthis -b "racfclassAct=Time,profileType=TSOCMD,cn=racf,o=site"
  "(objectclass=racfprogrammername)"
```

This example shows how the parameters to ldapsearch contain the host connection information and bind user crededentials alongside the Base DN. The command can be entered on an OMVS interactive session or invoked from a batch job using the BPXBATCH utility.

The &PREFIX.CNTL(LDAPTEST) job that was used for installation verification also uses the 'racfclassact=time' example because if that completes successfully it indicates that most of the components for using LDAPX with TDS are in place and functioning correctly.

Another member of &PREFIX.CNTL called LDAPTST2 is provided as an example of how streams of z/OS Unix commands can be put together and executed in order to make issuing commands like ldapsearch very much easier to achieve in batch. LDAPTST2 is provided purely as an example and a basis for tailoring in case it proves of some use in your setting. It is not designed to be used as-is, and it is the responsibility of the customer to ensure that it is suitable and safe to run in their own setting.

# Further Tailoring Considerations and Usage Notes

### LDAPX Configuration parameters

Another section of this document lists the LDAPX configuration parameters in detail and it is worthwhile looking through that to see if there are any settings which would better suit the way LDAPX is used in your environment. However, some of the ones you might do well to consider are listed here.

'**binduser**' - If there are restrictions on how system special RACF ids are used in your environment and/or what userids can manage particular resources it is possible to configure LDAPX to work with that by using the various '**binduser**' and associated parameters. There is discussion around that subject in the configuration parameters section of this document and it is also mentioned in the RACF section, and in comments in the LDAPRACF job.

'**tsocmd*dddname***' – When using the TSOCMD facility the only commands available by default are those that reside in system LPA and linklist libraries (e.g. TSO commands, IDCAMS commands, RACF commands, etc.). It is also possible to run in-house written EXECs, CLISTs, and modules by using the 'tsocmdSYSPROC', 'tsocmdSYSEXEC', and 'tsocmdISPLLIB' statements to specify a CLIST, EXEC, and load library dataset name respectively that will be allocated and made available to the TSOCMD process.

**'log'** – Several parameters which allow you to control the LDAPX log.

The '**MaxUser*xxxxxxx***', '**MaxGroup*xxxxxxx***', and '**MaxClass*xxxxxxx***' statements, where xxxxxxx is 'Standard' or 'Enhanced', control whether LDAPX is used to list those types of RACF profile instead of letting SDBM handle them. If SDBM handles them, the amount of profiles that can be returned for that type is limited to 4096.

'**AutoDatasetRefreshOff'** – By default a 'SETROPTS CLASS(DATASET) REFRESH' command is issued after every single process which makes a change to a profile of type DATASET. Depending on whether dataset profiles are being managed by LDAP and the frequency of any such changes this might be an overhead. In that case you may choose to have '**AutoDatasetRefreshOff'.** Note though that you then may need to arrange to issue the REFRESH command periodically to make the changes available.

### The TDS DEBUG Trace

This is a part of the TDS software as supplied by IBM but it is mentioned here as it is a very useful tool to know about. The LDAPX installation process even tailors the DSSRV proc to include comments about using the TDS DEBUG trace. One Identity Support personnel will often ask for a DEBUG trace to be switched on to capture events when trying to diagnose a reported fault.

This is a helpful tool as it shows a lot of detail of what is going on, but note that it can create a great deal of output and increase CPU usage, so it is best to leave the DEBUG trace disabled until it is required to help find the source of a problem. When the TDS (DSSRV) task is running, the DEBUG trace can be switched on and off dynamically by using the z/OS MODIFY command.

Different aspects of TDS can be traced by using different parameters on the debug command. The most commonly used parameters for finding problems with TDS using LDAPX is to trace the SDBM calls and the plug-in exit. This can be switched on dynamically by using the command:

**F DSSRV,DEBUG SDBM+PLUG**

…and stopped again by using the command

**F DSSRV,DEBUG OFF**

The trace data is written to the DSSRV started task output.

It is also possible to specify the debug trace be started when the TDS task is started. This is useful as it can capture both the TDS and the LDAPX configuration parameters. But it is best NOT to start the DSSRV task with the debug trace active on a regular basis, to limit the output and CPU usage as mentioned above. To start TDS and activate the debug trace in this way the command would look something like:

**S DSSRV,PARMS='-d SDBM+PLUG'**

The trace can be stopped dynamically by using the modify (F) command as shown above.

For further information on the TDS debug trace it is recommended to refer to the IBM TDS manual: 'IBM Tivoli Directory Server Administration and Use for z/OS'.

**LDAPX Execution Threads**

For the most part, the LDAPX code is 'single threaded' so as to avoid contention for resources. For this reason the default LDAPX tailoring sets the TDS configuration file (ds.conf) 'commThreads' parameter to the value 2. This may clash with requirements of installations who already use TDS and have set the value of commThreads higher.

In those cases it would be worth monitoring the systems in the first instance to see if changing the value to 2 has any adverse affect on the existing function. If it does have an adverse affect it could be increased by a small amount and see if that affects the running of LDAPX / SDBM. How the systems will cope depends entirely on the frequency and variety of requests in the system. If it proves difficult to balance the opposing needs it may be worth considering running a TDS task for each function so that tuning can be optimised for each.

Regardless of the commThreads setting, the fact that the LDAPX code is 'single-threaded' means that when LDAPX is processing a request, any others that arrive have to wait until the first is finished. The LDAPX Configuration parameter 'maxwait' defines how long (in seconds) such a request should wait before giving up. The default value of 37 should be suitable for most installations, but it may need to be increased if you know that there are regularly TSOCMD or DATASET processes which take longer to execute.

When the maxwait time is exceeded the LDAPX code just passes the request on to the TDS LDAP server without handling it itself. As the request is something that LDAPX *would* have processed it is very likely the TDS LDAP server will return an error to the client for that request. Therefore an observed increase in

the number of failed transactions might indicate that there are several requests arriving at LDAPX in a short time period, some of them are having to wait, and the maxwait time is being exceeded.

To find out if that is happening the TDS debug trace should be enabled, using the PLUG parameter to trace the plug-in exit events. If requests are having to wait the trace will contain regular messages like:

```
MUTEX WAIT errno=114 (count=xx)
```

.. where xx is a number which increments every second up to the maxwait value. If/when the maxwait time is exceeded you will also see the messages

```
MUTEX BUSY errno=114 after several attempts
Mutex already busy do just Return from the Exit
```

.. which indicates LDAPX will no longer handle that particular request, it will be passed through to TDS, and probably return an error to the client.

To reduce the number of failed requests you may simply try increasing the maxwait time, which should work in most cases. In addition, the frequency at which requests are being sent to LDAPX has a bearing on this, as does the complexity and execution time of any processes being run through the TSOCMD facility. So any measures that can be taken to improve those things will help the throughput that LDAPX can achieve and therefore reduce the likelihood of the MUTEX WAIT situation arising in the first place.

# LDAPXCFG Configuration File

## Introduction

Configuration parameters for the processing within the LDAPX plugin exit are required if DATASET profiles or TSO commands are to be managed, and are recommended but not strictly mandatory otherwise. They can be supplied in a file specified as ddname LDAPXCFG in the TDS started task JCL within your proclib concatenation (for instance by adding the ddname to proclib member DSSRV if the defaults were used when TDS was installed). The ddname should specify the PDS & member containing your parameters; the installation process inserts this into the customized copy of IBM's sample JCL (member DSSRV) prepared by the install in the …CNTL library (and in your PROCLIB library if you run PART3 of the install job).

For example: **`//LDAPXCFG  DD DSN=USER.PARMLIB(LDAPXCFG),DISP=SHR`**

The supplied LDAPXCFG member is heavily commented and it can be useful to see the statements in the context they are actually used as well as reading the information below.

## Syntax

a) The parameters are coded one per line starting in column 1 and are of the form
   Keyword=data  comments
b) A blank space indicates the end of the data for the line and anything following is taken to be comment.
c) Keywords are case insensitive.
d) Keywords not recognized are also taken as comments.

## Keyword options (shown in functional groups)

| **LICENSING** |
|---|
| **LicenseCode=nnnnn**<br><br>A code that is correct for your environment will enable licensable features and may be obtained from the Vendor. Without a correct code the exit will handle listing USER & GROUP profile names to overcome the 4096 limit within SDBM, this would be indicated by this JOBLOG message at startup time:<br><br>LDAPX running in basic mode on cpuid … to support more than 4096 users & groups<br><br>With a correct code all features are available. Licensed mode is indicated by this JOBLOG message:<br><br>LDAPX running in full extensions mode on cpuid … |

**SPECIFYING BIND USERS AND WHAT THEY CAN DO**

**binduser=uuu**

adds system special RACF userid 'uuu' to a list of bind users who can use the features of the exit. If **no** binduser keywords are supplied then **all system special RACF** users will have access to all features as long as no bind users are specified with the permit option (see below).

If **any** binduser keywords are supplied then **only** those specified users can have access to all features. Note that this means that even bindusers with RACF System Special authority need to be specified with a binduser= statement.

**FacilityPermit**

enables the use of the permit form of binduser keyword (see below).
This requires a profile ONEIDEN.LDAPX.PERMIT to exist in the FACILITY class in order to take effect, which may be created with a sequence of RACF commands as follows:

```
RDEFINE FACILITY ONEIDEN.LDAPX.PERMIT UACC(NONE)
PERMIT ONEIDEN.LDAPX.PERMIT CLASS(FACILITY) ID(LDAPSRV) ACCESS(READ)
SETROPTS RACLIST(FACILITY) REFRESH
RLIST FACILITY ONEIDEN.LDAPX.PERMIT ALL
```

Note that there needs to be at least one entry explicitly defined on the access list of the profile. It does not have to be LDAPSRV as in the above commands - the access list is checked simply for the presence of any RACFid. The access level of the entry is ignored.

**binduser=uuu,permit**

adds RACF userid 'uuu' to a list of bind users who can use the features of the exit. Userid 'uuu' does not need to have system special attribute
This requires an earlier FacilityPermit keyword to have enabled the permit keyword.
If any binduser keywords are supplied then only the specified users can have access to all features.

**useBinduser**

specifies that all TSO commands issued by the LDAPX plugin exit (including any generated by ldap change requests for dataset entries) will be performed under the authority of the userid supplied to TDS in the ldap bind credentials rather than the userid of the TDS server started task. To allow this the TDS userid must be defined in RACF as a surrogate for each bind user, using commands such are these…

```
RDEFINE SURROGAT BPX.SRV.binduser UACC(NONE)
PERMIT BPX.SRV.binduser CLASS(SURROGAT) ID(tdsuser) ACCESS(READ)
SETROPTS RACLIST(SURROGAT) REFRESH
```

…as documented by IBM in webpage at…
https://www.ibm.com/support/knowledgecenter/en/SSLTBW_2.1.0/com.ibm.zos.v2r1.bpxa500/suc.htm

Note that this keyword alone will not affect lists of profile names returned for requests from non-special bind users - if you wish to enforce strict RACF scope rules on those lists then specify keyword ScopeCheck too (or instead, same effect).

Without the useBinduser keyword the default action is to issue all commands using the authority of the userid running the TDS started task, which after a standard install will be LDAPSRV.

**Check=uuu**

Will indicate if bind user uuu is found in the binduser tables. Specify Check=*all* to list all entries in binduser tables.

**CheckRACF=ppp**

Will check and trace the existence of a RACF profile ppp in the FACILITY class. Ids with direct access permission (not via groups) are shown. This is used internally for FacilityPermit and is otherwise only for testing. Note that only the specific profile mentioned above will enable the permit form of binduser.

**ScopeCheck**

Required if keyword UseBindUser is in effect, especially if there is an intention for bind users not to be RACF special users. Signifies that scope of authority of non-special bind users will be checked when generating any of the enhanced listings that allow for more than 4096 names of user, group, general resource, or dataset profiles. This keyword has no effect for special bind users since for them scope checking is irrelevant, they have access to all profiles and so their lists are always generated purely from 'racroute extract next' APF calls. Specifying this keyword will also force processing to act as if keyword UseBinduser has been specified.

Previous releases of LDAPX allowed parameter ScopeCheck=n as well as ScopeCheck. Note that this format is still accepted for compatibility, but the use of ScopeCheck=1 or ScopeCheck=2 are not recommended and their use can have unpredictable results.

**ScopeCheckSearch=search**

specifies the TSO command or clist or rexx exec to be used for scope checking.

It will be called with parameters suitable for the RACF TSO command 'SEARCH', such as CLASS(USER|GROUP|DATASET) and optionally NOMASK|FILTER(xxx), and also GENERIC|NOGENERIC with CLASS(DATASET). Note that if the value represents a clist or rexx rather than a command, the tsocmdSYSPROC and/or tsocmdSYSEXEC keywords may also be required, in the same way as they would if issued via an LDAP search for DN racfClassAct=... .

Hardcoded default value, which should normally not need to be overridden, is **search**, meaning the RACF TSO command.

**HideDiscreteDatasets**

specifies that discrete dataset profile names are to be omitted from lists returned to non-special bindusers.

The search method does not provide volume information of discrete dataset profiles, thus the DN, which would contain it, cannot be constructed correctly, which means they should be omitted. This keyword has no effect for special bind users since for them scope checking is not performed. (Fix LDF2103 erroneously documented this keyword as OmitDiscreteDatasets).

Not set by default.

**SCOPE OF WHAT RACF PROFILES LDAPX SHOULD ACT ON**

**MaxGroupEnhanced | MaxGroupStandard**

If neither of above is specified MaxGroupEnhanced is default. Uses LDAPX to list groups so all entries returned even if more than 4096. Specify MaxGroupStandard to use SDBM to list groups (so result will be limited to 4096).

**MaxUserEnhanced | MaxUserStandard**

If neither of above is specified MaxUserEnhanced is default. Uses LDAPX to list users so all entries returned even if more than 4096. Specify MaxUserStandard to use SDBM to list users (so result will be limited to 4096).

**MaxClassStandard | MaxClassEnhanced | MaxClassEnhanced=xxxx | MaxClassEnhanced=xxxx***

If none of above is specified the default is MaxClassStandard which means lists of profiles in each class will be provided by SDBM, so result will be limited to 4096 lines. If MaxClassStandard is specified then any MaxClassEnhanced statements earlier in the configuration file will be ignored.

Specifying MaxClassEnhanced is the same as specifying MaxClassEnhanced=* which means that LDAPX facilities are used to list profiles of **all** RACF classes so results are not limited to 4096 items. If you need to use LDAPX facilities to manage RACF classes that have >4096 profiles defined it is highly recommended to use the MaxClassEnhanced=xxxx format of the statement as described below to pick out the classes for processing explicitly and avoid unnecessary processing of the other classes.

Use MaxClassEnhanced=xxxx to specify a particular RACF class that LDAPX should handle. This statement can be repeated in order to specify all such classes that LDAPX should handle. The '*' suffix format can be used to select all classes that begin with the string 'xxxx'.

If using MaxClassEnhanced=xxxx, xxxx cannot be 'USER', 'GROUP', 'DATASET', or 'TSOCMD'

| CHARACTER TRANSLATION |
| --- |
| **AsciiExtended**<br><br>Translation between ASCII & EBCDIC of DNs and attribute values processed by the exit uses an internal table. This keyword enables that translation to support extended ASCII, that is: to support the recognition of ascii codes in the range hex 80 to FF that are transmitted in 2-byte UTF-8 form. Note that an extended ascii value cannot be placed into an LDAP attribute defined as IA5 (syntax 1.3.6.1.4.1.1466.115.121.1.26) and that without the PTF for IBM APAR OA47787 which redefines 14 RACF attributes as 'directory string' (syntax 1.3.6.1.4.1.1466.115.121.1.15) the racfprogrammername attribute (used by LDAPX for output from TSO commands) cannot contain extended ascii. Thus any lines of TSO command output that contain characters that translate to extended ascii cannot be returned unless the IBM fix is also applied. The PTFs are UA77810 & UA77811 for z/OS 1.3 & 2.1 according to this link http://www-01.ibm.com/support/docview.wss?uid=isg1OA47787<br><br>**AsciiStandard**<br><br>Translation between ASCII & EBCDIC of DNs and attribute values processed by the exit uses an internal table. This keyword disables that translation from supporting extended ASCII and is the default.<br>Any 2-byte UTF-8 representations of extended ascii values (hex 80-FF) that are translated by the exit will result in 2 ascii characters which are acceptable to an attribute defined as IA5 (for instance the output of a TSO command into programmername prior to the APAR mentioned above).<br><br>**codeAscii=aaa**<br><br>Specifies the name of an IBM codeset for ASCII. For testing purposes only and in conjunction with codeEbcdic keyword and not a supported feature of the product.<br><br>A list of IBM codesets is given at<br>https://www.ibm.com/support/knowledgecenter/SSLTBW_2.1.0/com.ibm.zos.v2r1.cbcpx01/loccomp.htm<br><br>Default is Ascii=ISO8859-1<br><br>**codeEbcdic=eee**<br><br>Specifies the name of an IBM codeset for EBCDIC. For testing purposes only and in conjunction with codeAscii keyword. Specifying this keyword will refresh the internal ascii to/from ebcdic translation tables with single-byte translation obtained from performing IBM iconv conversions using the specified codesets. Thus an alternative to the default internal translation may be tested.<br><br>A list of IBM codesets is given at<br>https://www.ibm.com/support/knowledgecenter/SSLTBW_2.1.0/com.ibm.zos.v2r1.cbcpx01/loccomp.htm |

Default suggested is Ebcdic=IBM-1047 (the internal tables are predefined and not necessarily to this specification above translations to ascii 7F)

**codeList**

Lists the current contents of the internal ascii to/from ebcdic translate tables. For testing purposes only and not a supported feature of the product.

**TSOCMD, DATASET AND CLASS PROFILE SUPPORT**

**scriptsoPath=ppp**

specifies the full pathname ppp of the script used by the exit to issue TSO commands, both for arbitrary TSOCMD requests and for internal commands to change DATASET profiles.

Default is scriptsoPath=/usr/oneiden/scriptso

**tsocmdPath=ttt**

specifies the full pathname ttt of the tsocmd script used by the exit to issue TSO commands indirectly from within the script specified by scriptsoPath. For normal running the tsocmd Unix Shell Command provided by IBM should be used, and is normally in /bin/tsocmd although before z/OS 1.10 could have been downloaded to a different directory.

The default value, which should normally not need to be changed, is tsocmdPath=/bin/tsocmd

**tsocmdxxxxxxx=**

The tsocmd… keywords are used to name the datasets that will be used in 'export' environment set statements issued internally before invoking USS command **tsocmd** to issue the required tso command inside a temporary ispf shell. There are 4 of them:

**tsocmdSYSPROC=ccc**

…specifies the dataset name of the PDS to be used as source of clists that may be issued via the TSOCMD feature.

**tsocmdSYSEXEC=rrr**

…specifies the dataset name of the PDS to be used as source of rexx execs that may be issued via the TSOCMD feature.

**tsocmdISPLLIB=sss**

…specifies the dataset name of the PDS to be used as source of user command programs (i.e. executable load modules) that may be issued via the TSOCMD feature.

**tsocmdSYSUADS=ddd**

…specifies the dataset name of the PDS to be used as User Attributes DataSet for any commands that may require it and be issued via the TSOCMD feature (primarility for ACCOUNT command which can manage TSO users' attributes if any are not defined in RACF). Default which should normally not need to be changed is tsocmdSYSUADS=SYS1.UADS unless changed during installation

**scriptso=*export***

constructs a parameter string from the above 4 keywords for specifying the libraries for TSO commands. Must appear after all the required libraries have been specified, i.e. after the 4 above tsocmdxxxxxxx statements. If the value *export* is not specified exactly then the library keywords tsocmdxxxxxxx have no effect.

**tsocmdESCchars=ccc**

specifies a string of individual characters which, if they appear in a TSO command, will be automatically escaped by inserting a preceding '\' backslash before invoking the internal USS script. The string should include characters $"\ because otherwise $ would be interpreted by USS as the start of a variable name with no value and truncate the command, " would be interpreted as an unintended string delimiter, and a single \ would be taken as the start of an unintended escape sequence. Automatically escaping those characters just before invoking the script enables them to be interpreted as part of the TSO command.

The hardcoded default string, which should not normally need to be overridden, is $"\

**tsocmdWrapper=ldaprc %s**

specifies that each tso command (or clist or rexx) issued by the plug-in is to be passed as a parameter to a 'wrapper`, which is intended to be a rexx exec or clist that issues the command and afterwards outputs the command's return code in a format that can be recognised. This keyword enables LDAPX to know exact return codes from commands it issues (without it only the text output, if any, would be available). Specifying the use of the wrapper concept avoids the default behaviour of having to assume 'no TSO output messages' means the return code was 0 (success) and 'some TSO output' indicates errors (which works for RACF updating commands but may not be appropriate when the 'command' is actually a clist or rexx or a TSO command that lists something).

The value of this keyword is terminated by 3 consecutive blanks (not 1) in order to allow the string to contain blanks (so there must be at least 3 spaces between the value and any comment on the same line).

If you wish to indicate the name is a clist or rexx to avoid searching command program libraries (i.e by % as first character) it will have to be doubled (%%) since a single % will indicate internal sprintf formatting (as used to dynamically convert the %s into the command-as-a-parameter). The value is intended to contain %s which will be replaced by the command being issued, thus a value of 'ldaprc %s' (without the

quotes) would pass the command as one string to a rexx exec named ldaprc. It is intended that the rexx outputs the return code enclosed between delimiter strings specified by keyword tsocmdRCstring, whose default is ==LDAPRC==, so using that, the rexx would indicate a return code of 123 by adding ==LDAPRC==123==LDAPRC== to the TSO output.

A sample rexx wrapper with detailed comments and optional echo debugging statements is supplied in ...CNTL(LDAPRC). It would need to be copied to your PDS library mentioned on keyword tsocmdEXEC or tsocmdSYSPROC.

A simpler version follows here:

```
/* a rexx exec */
trace 'o'                /* o (notrace) or r (trace) */
parse arg allparms       /* don't uppercase */
allparms
say '==LDAPRC=='rc'==LDAPRC=='
```

Not set by default.

**tsocmdRCstring=ccc**

this keyword (or its default value) is used together with the tsocmdWrapper keyword to enable LDAPX to base its indication of success or failure of each TSO command (or rexx or clist) on the actual return codes from TSO, rather than on the mere presence of any output messages, which are assumed to indicate errors. This applies to internal commands issued during processing of dataset profiles, in addition to TSO commands requested explicitly, for which the detected return code is made available to the LDAP client in an attribute named by keyword tsocmdRcAttribute or its default. If this tsocmdWrapper/tsocmdRCstring feature is not used then the presence of one or more messages in the output of a TSO command (or rexx or clist) is assumed to indicate failure, which, for clists & rexx execs especially, may well not be correct. See the section for keyword tsocmdWrapper for further explanation.

The value of this keyword is terminated by 3 consecutive blanks (not 1) in order to allow the string to contain blanks (so there must be at least 3 spaces between the value & any comment on the same line).

Default value is ==LDAPRC==

**tsocmdRcAttribute=ccc**

specifies the name of the attribute to be used to return TSO return code information to the LDAP client. This only occurs if keywords tsocmdWrapper & tsocmdRCstring have appropriate values and the named attribute is in the schema. The default value is the name of an IBM RACF attribute so will always be in the schema but could be misleading if used, whereas the recommended value is racfTsoCmdRc, which is one of the attributes added when the schema was modified during the install of ldapx 2.2 or later by the job supplied in ...CNTL(LDAPMOD) and LDIF data in ...REPRO(LDIFSCHN).

Default value is racfPasswordInterval

Recommended value is racfTsoCmdRc

**tsocmdMSGstring=ccc**

specifies whether all or only one, of any text lines that may be output by internal TSO commands generated for dataset-related LDAP add/modify/delete requests, are to be logged and returned as the LDAP result error message. Value should either be ALL (meaning any/all output from TSO for add/mod/del is logged & returned as an error message & causes the add/mod/del to be considered as having failed) or a 3 digit number corresponding to the single line of output, if present, to use. For instance 002 could be specified in order to only return the 2nd line (if it exists in the TSO output) because the 1st perhaps was a customer echo statement inserted into the internal USS scriptso that did not start with the string specified on a tsocmdECHOstring= keyword.

Hardcoded default is ALL.

**tsocmdECHOstring=echo from scriptso**

specifies what string, if any, at the start of a line of output from the internal USS script (scriptsoPath), causes that line to be ignored when considering output from internal or arbitrary TSO commands. The value of this keyword is terminated by 3 consecutive blanks (not 1) in order to allow the string to contain blanks (so there must be at least 3 spaces between the value & any comment on the same line). Ignored lines are not included in the count for arbitrary TSO commands (reported as LN:nnn in TSO:USERLN log messages) nor is their text logged or included in error messages returned to the requestor in the LDAP result or in attributes returned for profiletype=TSOCMD searches.

This provides a way for including an echo statement in scriptso, and/or a say or write statement in a rexx or clist, to indicate in the trace that scriptso and/or the rexx has actually run, without the extra output altering data returned to the LDAP requestor.

There is no default string. Maximum length 19.

**LOGGING OPTIONS**

**logFilename=/usr/oneiden/ldapx.log**

specifies the basic name for ldapx log files, overriding the optional parameter on the TDS plugin statement in /etc/ldap/ds.conf which LDAPX would otherwise use as the name for its log file. A timestamp suffix of _yymmdd_hhmm will be inserted before the 1st dot (or at end), unless keyword logSwitch=0 is specified. The suffix enables a different log files to be started after each TDS restart or after each log switch caused by non-zero logRollSize or logRollFreq values.

A default of ldapx_plugin_log.txt is used if this keyword and the parameter on the ds.conf plugin statement are both omitted.

**logSwitch=1 | logSwitch=0**

Specifies whether log switching should occur. Using logSwitch=0 disables log switching, overriding the other log management parameters below, so the same log file will be used all the time. Note that the log file can grow very large so using logSwitch=0 is not recommended unless external processes to manage the file are available / preferable.

The default value is logSwitch=1 , which enables use of the logRollSize or logRollFreq keywords to manage the log file.

**logRollSize=nnn**

specifies the size (in bytes) that will cause a switch to a new log file (unless logSwitch=0 is specified). Before each message is written the current size of the file is compared, and if larger then the message will be written to a file containing a new timestamp suffix in its name, and that log file will become the current one.

Default is 500000000 (500M).

**logRollFreq=nn**

specifies the frequency of log file switching, as the length of timestamp to compare. Each log message is timestamped, if the beginning of the timestamp is different to that of the previous message, within the compared length, then the message will be written to a file containing a new timestamp suffix in its name, and that log file will become the current one.

The message timestamp is of the form 'Mon Jul 16 13:45:59 2018', so logRollFreq=16 means switch if the minute is different, 15 means if the ten minute digit is different, 13 the hour, 12 the tens of hours digit (i.e. three times a day), 10 means each day. The timestamp suffix for each log filename is _yymmdd_hhmm corrresponding to the time of the first message within it. Empty files are not produced, i.e. a new file is only started when there is a message to write, so there will be no files with timestamps of long inactive periods (for instance if logRollFreq=10 and there was no activity on a Sunday then there will be no empty log file for that day).

Hardcoded default is 10, a different log file for each day (assuming there was something to log).

**logFlushSize=nnn**

specifies the incremental size (in bytes) of the current log file before flushing buffers by closing and reopening it to ensure all earlier messages are physically written and visible if subsequently browsing or copying the file. The size is checked after each message is output, so a value of 1 would reopen the log after every message, for instance to enable immediate browsing during tests, but could involve i/o inefficiencies.

Hardcoded default is 10000

**logFlushFreq=nnn**

specifies the frequency of flushing log file buffers, as described in logFlushSize, in the same timestamp length way as used for logRollFreq.

16 = each minute, 15 = each ten minutes, 13 = each hour, etc. No flushing when no messages. Messages output within the specified period prior to the last log message may not be physically written until a future message gives another flushing opportunity. For repetitive functional test & browse sequences where seeing the latest message in the log file is important, and efficency is not so, use logFlushSize=1 as described above (which will close & reopen the log file after each message so not advised for production).

Hardcoded default is 16 (meaning messages not yet physically in the file would be no older than a minute before the time of the last activity).

**logTSO=n**

specifies whether to log all text lines or only the first, of output from arbitrary TSO commands (as issued for racfClassAct=...,profiletype=TSOCMD searches).

1=yes, log all lines. 0=no, log only the first line. The number of lines produced for such commands is always logged regardless of this value.

Default is 1, log all lines.

**MISCELLANEOUS**

**AutoDatasetRefreshOff**

specifies that the command 'setropts generic(dataset) refresh' will not be issued automatically after each dataset profile change caused by an ldap request affecting a dataset entry (dn starting with racfDataset= or racfPermitId=). Note that changes to dataset profiles may not have a security effect on tasks that are already running until the refresh command is issued on the system being managed (see IBM help for for details, for instance as output by tso command 'help altdsd').

By default the refresh command is issued immediately after each implicit addsd, altdsd or permit command issued when processing ldap change requests for datasets.

**MaxWait=nn**

Specifies the number of seconds that LDAPX will wait for plug-in thread control. Default is 37.

# RACF permission considerations if TDS stc or bind user does not have SYSTEM SPECIAL

### General considerations

There are two modes of operation to consider when TDS is running with the SDBM backend and the LDAPX plugin exit. Each mode can operate under a different user. The plugin exit is one mode and operates as an extension of the TDS task, the SDBM backend is the other mode and switches to a bind user before calling r_admin (IRRSEQ00) to issue RACF commands. Thus all of the considerations below may apply to either the TDS task user (for issuing RACF or other TSO commands in order to provide LDAPX exit features) or to the bind user (for LDAP requests handled by SDBM).

If both users have the SPECIAL attribute (as can be set by command ALTUSER <user> SPECIAL) then almost all of the considerations below will not be required, and the sample commands supplied in the job in …CNTL(LDAPRACF) should be sufficient to enable a TDS-with-LDAPX system to operate. If SPECIAL is to be avoided, then your site RACF Security Administrator may have alternatives or recommendations beyond those suggested here. The following list may not be exhaustive…

### SDBM backend

For the SDBM backend to use the R_admin callable service (IRRSEQ00) to internally issue commands corresponding to the RACF operations requested via LDAP, if the bind user does not have SYSTEM SPECIAL then they will require READ permission on the following profiles:

```
CLASS PROFILE
FACILITY IRR.RADMIN.ADDUSER
FACILITY IRR.RADMIN.ALTUSER
FACILITY IRR.RADMIN.CONNECT
FACILITY IRR.RADMIN.DELUSER
FACILITY IRR.RADMIN.PASSWORD
FACILITY IRR.RADMIN.REMOVE
```

Note it is required to use the full command name as shown in the examples above when you define the resource, and that the plugin library and module must be program controlled (a USS concept) and run from an APF authorized library which can be accomplished by an extattr +ap command that may be executed during installation.

The RACF user ID that binds to TDS to request SDBM functions may also require specific privileges. For example, if the same bind user administers all users in the RACF database, they must operate with the SYSTEM SPECIAL RACF attribute. If the bind user administers only a portion of the RACF database, then they would need to be associated with a group assigned GROUP SPECIAL privileges for the portion of the database they administer.

**Other resources**

Other RACF resources that may require consideration are:

**FIELD class profile USER.segment.**, with UPDATE**
FIELD class profiles may be required when the user does not have the SYSTEM SPECIAL attribute.

**FACILITY class profile STGADMIN.IGG.DEFDEL.UALIAS, with READ**
The STGADMIN.IGG.DEFDEL.UALIAS might be required by the TDS stc user if catalog aliases are created by commands issued via the LDAPX plugin TSOCMD feature.

**FACILITY class profile IRR.PASSWORD.RESET, with UPDATE**
IRR.PASSWORD.RESET is required if the effective RACF ID that changes passwords or pass phrases does not have the SYSTEM SPECIAL RACF attribute.

**The STGADMIN.IGG.DEFDEL.UALIAS might be required if catalog aliases are**
STGADMIN.IGG.DEFDEL.UALIAS is required if your user exits create or delete catalog aliases and the effective RACF ID does not have MCAT update authority.

**UNIXPRIV class profile SHARED.IDS, with xxxx access**
The bind user or TDS user requires access to this profile if SDBM or the plugin (via TSOCMD) creates RACF IDs with OMVS segments where duplicate UIDs are created.

**CLAUTH with class of USER**
CLAUTH of USER is required if SDBM or the plugin (via TSOCMD) creates RACF users, when the creating ID does not have SYSTEM SPECIAL.

## Permissions for LDAPX

Specific permissions are required for the TDS task user in order for the exit to be able to run TSO commands. The TDS user will require a minimal TSO segment definition (SIZE & UNIT are sufficient), and execute access to the directory containing the scriptso script is required if TSO command or DATASET features are being used, plus access to any datasets specified for TSO or used by clists. Also the above considerations for a bind user performing SDBM requests also apply to the TDS task if DATASET features of LDAPX are used since they run using the authority of the TDS task.
These RACF commands are suggestions and have been sufficient in a simple RACF test environment. The list includes some that may have already been addressed or you have decided to modify for the considerations above or when TDS was originally configured in which case not all will be required.
This list is also supplied in the …CNTL(LDAPRACF job:

```
 /* IBM Tivoli Directory Server Administration and Use for z/OS
 /* Document Number SC23-5191-07

 /* 1.6.2 Requirements for a user ID that runs the LDAP server
 /* You can use the RACF commands in the following example to define
 /* the userID that runs the LDAP server (NOPASSWORD so cannot logon)
 /* (substitute appropriate values for UID and GID)
ADDGROUP LDAPGRP SUPGROUP(SYS1) OMVS(GID(2))
ADDUSER LDAPSRV DFLTGRP(LDAPGRP) NOPASSWORD OMVS(PROGRAM('/bin/sh'))
```

```
ALTUSER LDAPSRV TSO(SIZE(2048) UNIT(SYSALLDA)) SPECIAL OMVS(UID(0))
 /* The following RACF commands give the LDAP server access to
 /* the Workload Manager (WLM).
RDEFINE FACILITY BPX.WLMSERVER UACC(NONE)
PERMIT BPX.WLMSERVER CLASS(FACILITY) ID(LDAPSRV) ACCESS(READ)
SETROPTS RACLIST(FACILITY) REFRESH
 /* Grant control access to the SUPERUSER.FILESYS profile in
 /* the UNIXPRIV class.
 /* Control access to the SUPERUSER.FILESYS profile might not be
 /* wanted as it grants the LDAP server user ID access to all files
 /* and directories in z/OS UNIX System Services.
 /* (see z/OS UNIX System Services Planning for more information)
 /* Note: this was found not to work so we use OMVS(UID(0)) instead.
 /*RDEFINE UNIXPRIV SUPERUSER.FILESYS UACC(NONE)
 /*PERMIT SUPERUSER.FILESYS CLASS(UNIXPRIV) ID(LDAPSRV) ACCESS(CONTROL)
 /*SETROPTS CLASSACT(UNIXPRIV)
 /*SETROPTS RACLIST(UNIXPRIV) REFRESH

 /* 1.6.2.1 Additional setup when using SDBM
 /* If you plan to use an SDBM backend, the following RACF commands
 /* must be entered to set up the user ID that runs the LDAP server:
RDEFINE FACILITY IRR.RUSERMAP UACC(NONE)
PERMIT IRR.RUSERMAP CLASS(FACILITY) ID(LDAPSRV) ACCESS(READ)
SETROPTS RACLIST(FACILITY) REFRESH

 /* 1.6.2.3 Additional setup for sysplex (optional)
 /*RDEFINE FACILITY GLD.XCF.GROUP.LDAP UACC(NONE)
 /*PERMIT GLD.XCF.GROUP.LDAP CLASS(FACILITY) ID(LDAPSRV) ACCESS(READ)
 /*SETROPTS RACLIST(FACILITY) REFRESH
 /* 1.6.2.5 Additional setup for generating audit records (optional)
 /*RDEFINE FACILITY IRR.RAUDITX UACC(NONE)
 /*PERMIT IRR.RAUDITX CLASS(FACILITY) ID(LDAPSRV) ACCESS(READ)
 /*SETROPTS RACLIST(FACILITY) REFRESH

 /* 1.6.2.7 Additional setup when defining administrative roles in RACF
 /* If the administrative group and roles are to be defined in RACF,
 /* the BPX.SERVER profile in the FACILITY class is required
PERMIT BPX.SERVER CLASS(FACILITY) ID(LDAPSRV) ACCESS(UPDATE)
SETROPTS RACLIST(FACILITY) REFRESH

 /* 1.10.2.1 Defining the started task for the LDAP server
 /* To define the started task for the user ID you created, you can
 /* use the following RACF commands
RDEFINE STARTED DSSRV.** STDATA(USER(LDAPSRV))
SETROPTS RACLIST(STARTED) REFRESH

 /* 1.12.4.1 Using RACF key rings (only required for SSL)
RDEFINE FACILITY IRR.DIGTCERT.LIST UACC(NONE)
RDEFINE FACILITY IRR.DIGTCERT.LISTRING UACC(NONE)
PERMIT IRR.DIGTCERT.LISTRING CLASS(FACILITY) +
  ID(LDAPSRV) ACCESS(CONTROL)
PERMIT IRR.DIGTCERT.LIST CLASS(FACILITY) ID(LDAPSRV) ACCESS(CONTROL)

 /* Security Server RACF
 /* Command Language Reference - Document Number SA22-7687-12
```

```
 /* 5.46 RALTER (Alter general resource profile)
 /* IBM Tivoli Directory Server for z/OS - Redbook SG24-7849-00
 /* 12.3.2 Setting up the HCD LDAP plug-in
 /* If RACF program control is active the plugin STEPLIB needs adding:
 /* (or use SYS1.LINKLIB which may already be in program control)
RDEFINE PROGRAM ** UACC(READ)
RALTER PROGRAM ** UACC(READ) ADDMEM('USER.LINKLIB'/'ZDSYS1'/NOPADCHK)
SETROPTS WHEN(PROGRAM) REFRESH
SEARCH CLASS(PROGRAM)
 /* This is required to allow plugin to use tsocmd for RACF commands
ALTUSER LDAPSRV TSO(SIZE(2048) UNIT(SYSALLDA)) SPECIAL
 /* These may be required, if you wish to submit jobs or mvs commands.
 /* Other PERMITs usually given to your TSO users may also be required
 /* depending on site configuration and intended usage.
 /* If you have a standard method for creating a TSO users you may
 /* prefer to use that and then run (or rerun) this job.
 /*PERMIT JCL  CLASS(TSOAUTH) ID(LDAPSRV) ACCESS(READ)
 /*PERMIT OPER CLASS(TSOAUTH) ID(LDAPSRV) ACCESS(READ)

 /* The following create a sample RACF userid for use in bind
 /* credentials used by LDAP clients.  For normal use this can be,
 /* but does not need to be, the same as the AdminDN userid
 /* mentioned in /etc/ldap/ds.conf
 /* However, the bind user specified when running ldapmodify to extend
 /* the schema at the end of this install process for the plugin, does
 /* have to be the one mentioned as AdminDN in /etc/ldap/ds.conf
 /* (otherwise ldapmodify fails with "Insufficient access: R003070")
 /* Important: change the password after running ...CNTL(LDAPMOD)!
 /* Note requires uid(0) to enable ldapmodify of schema
DELUSER LDAPADM
ADDUSER LDAPADM DFLTGRP(LDAPGRP) SPECIAL OMVS(UID(0))
ALTUSER LDAPADM PASSWORD(NOTTHIS) NOEXPIRE
LISTUSER LDAPADM TSO OMVS

 /* List all SDBM LDAP related items
SEARCH CLASS(FACILITY)
SEARCH CLASS(UNIXPRIV)
RLIST FACILITY BPX.DEAMON             ALL
RLIST FACILITY BPX.SERVER             ALL
RLIST FACILITY BPX.WLMSERVER          ALL
RLIST FACILITY GLD.XCF.GROUP.LDAP     ALL
RLIST FACILITY IRR.DIGTCERT.LIST      ALL
RLIST FACILITY IRR.DIGTCERT.LISTRING  ALL
RLIST FACILITY IRR.RAUDITX            ALL
RLIST FACILITY IRR.RUSERMAP           ALL
RLIST UNIXPRIV SUPERUSER.FILESYS      ALL
RLIST PROGRAM  **                     ALL
RLIST STARTED  DSSRV.**   STDATA      ALL
LISTUSER LDAPSRV TSO OMVS
LISTUSER LDAPADM TSO OMVS
RACDCERT ID(LDAPSRV) LISTRING(*)
RACDCERT ID(LDAPSRV) LISTMAP
RACDCERT ID(LDAPSRV) LIST
```

# Appendix A – LDAPX Release History

***Changes since release 2.1 include:***

a) Return LDAP result code 32 'not found error', not 0 'success', when a search for one or more DATASET or racfPermitId entries finds none.

b) Return LDAP result code 21 'syntax error', not 0 'success', when an internal TSO command outputs an error message, for instance due to a mistranslated DN or non-existent profile or lack of authority, during processing of an LDAP request to change a dataset entry. Also return the message, for example:

   i. ICH22005I NOT AUTHORIZED TO ALTER ADCDB.AL.*
   ii. IKJ56702I INVALID UNIVERSAL ACCESS, ZAQ (ie invalid)
   iii. ICH22001I ZADCDB.AL.** NOT DEFINED TO RACF
   iv. ICH22006I OWNER SPECIFIED NOT DEFINED TO RACF

c) Include processing for 2nd and subsequent attributes supplied in one LDAP modify request, for instance to update both instdata & uacc when new values are supplied in one modify request from ldp.exe or an LDIF file.

d) Correct a problem whereby a dataset entry representing a RACF generic dataset profile not containing generic-indicating characters such as * or % could not be found by a base search or update request, despite being returned in a non-base search for a list of all dataset entries.

e) Prevent LDAP requests for a profile name of ** being expanded by USS into a list of unix files, by inserting USS command 'set -o noglob' in the internal script.

f) Use the library names from LDAPXCFG keywords tsocmdSYSPROC & tsocmdSYSEXEC etc to override hardcoded defaults in the USS script without an unintended requirement to also specify a value for keyword tsocmdWrapper.

g) Prevent a '$' dollar character, in any attribute value destined to appear in an internal TSO command, from causing the value to be truncated due to $ being the prefix for unix script variables (unless supplied as \\$). Now a '\' backslash escape character is inserted before '$' characters, immediately prior to invoking the internal script. Note $ is not a char that requires RFC4514 escaping for LDAP transmission from the client. Characters " and \ also required pre-script escaping in order to be allowed in parameters. This change is externalised as an optional new keyword in LDAPXCFG of tsocmdESCchars= with default value $"\ supplied internally.

h) When a RACF dataset profile's UACC is NONE, correctly display the value of the corresponding LDAP dataset entry's racfUacc attribute as NONE, not EXECUTE which had been due to an erroneous IBM manual page.

i) Several Logging improvements: more info per TSO command, a timestamp prefix on each line, internal TraceMsg() instead of many TRACE macros, add LogSwitch() for new file per timestamp or

size, add flush buffers LogReopen() for instant viewability, each log file has start time in its file name as suffix _yymmdd_hhmm, base log filename on ds.conf plugin statement can be overridden by new keyword pluginLogFile= in LDAPXCFG.

j) Diagnostic tracing capability for the internal script by adding new keyword tsocmdECHOstring=

k) Consistent use of volume in DN to differentiation between discrete & generic datasets (complete management for discrete datasets remains unsupported)

l) Support for restricted view of dataset profiles by a non-special binduser with new keyword ScopeCheck

m) Larger internal buffers

n) Allow co-existence on same TDS of RACF dataset DNs (starting racfDataset= or racfPermitId= and ending with the SDBM suffix) with DNs starting racfDataset= etc but intended for a different backend (eg LDBM) due to not ending with the same (SDBM) suffix as the pre-authenticated (by TDS) binduser.

o) Add support for more than 4096 profiles within a general resource class, triggered by new MaxClassEnhanced= keywords. (Except for profiletype=CONNECT – see below).

p) Re-allow fixed log file name without a timestamp, so can also support dummy pluginLogFile=/dev/null or DD:ddname, triggered by keyword pluginLogUnique=0 or LogSwitch=0.

q) Increase the maximum supported length of general resource profile names from 48 to 246 which is the IBM maximum.

r) Skipped the processing of MAXCLASSENHANCED for the 'profileType=CONNECT'. This profiletype will be returning maximum 4096 results.

***Changes since release 2.0 include:***

a) Error with the permit form of binduser configuration where sometimes users not checked correctly for 'SPECIAL' authority before issuing TSO Command

b) Correct error where some special characters (those that required 'escape encoding' to more than one byte) in command string were not processed correctly

c) New feature where optionally TSO Command can be issued under the authority of the userid supplied to TDS in the ldap bind credentials rather than the userid of the TDS server.

***Changes since release 1.1 include:***

a) No artificial or safety limits are imposed when LDAPX lists the names of all USER, GROUP or DATASET profiles.

b) A simple filter of the form racfid=abc* is supported for USER and GROUP searches.

c) ASCII/EBCDIC translation of $ (dollar) is corrected.

d) Configuration settings for LDAPX may be specified, in a file specified as optional DD statement in the TDS JCL.

e) IBM-supplied EBCDIC codesets can be used in place of the internal translation tables, see configuration keyword codeEBCDIC later in this document.

f) All Latin-1 extended ascii (hex 00-FF) & corresponding 2-byte UTF-8 representations can be supported. There are pre-requisite IBM PTFs and a configuration option for LDAPX, see configuration keyword AsciiExtended.

g) LDAPX can be configured to provide its features to only specified bind users. The default remains that the features are available to all RACF system special users.

h) A license is required to enable processing for DATASET profiles & for issuing TSO commands.

i) TSO commands may contain the '!' exclamation mark character since it is no longer required (but can still be supplied) as a delimiter at the end.

j) The full path of the USS script by which TSO commands are issued is configurable in the file, so the script can be placed in a directory of your choice and need not be filename scriptso.

k) The MVS dataset names for TSO libraries such as SYSPROC & SYSEXEC containing the clists & rexx execs required for commands that are to be issued, can be changed more easily, in the configuration file.

l) The LDAPX install instructions indicate sections to omit or perform manually when adding LDAPX to existing TDS systems as opposed to when TDS has not previously been installed, and lists many RACF considerations which may be necessary if LDAPX is restricted to running without access to special userids.

# Appendix B – LDAPX SMP/E Installation

There is no intention to supply updates to LDAPX as SMP PTFs so there is no technical reason to install LDAPX using SMP/E facilities. However, if you do wish to use SMP/E please see the following notes for guidance.

The @README job needs to be tailored for PART1 processing so that it allocates some datasets to act as SMP/E 'relfiles' into which the supplied product materials are loaded. To do this a JCL SET statement is needed to set the variable PRESMP to a string valid as a dataset qualifier. The job as supplied contains the following lines:

```
//          SET PRESMP=''                NO EXTRA PREFIX FOR NON-SMP
//*          SET PRESMP='.RELFILE'        EXTRA PREFIX REQUIRED FOR SMP
```

…so a simple and valid course of action would be to make the first of those lines a comment and uncomment the second one, i.e.:

```
//*          SET PRESMP=''                NO EXTRA PREFIX FOR NON-SMP
//          SET PRESMP='.RELFILE'        EXTRA PREFIX REQUIRED FOR SMP
```

If not using the supplied value of '.RELFILE' remember to include a dot at the start of the chosen string.

Once PART1 has been run the relfiles will be created which contain the various product distribution components. These are used by jobs which run SMP/E utilities to populate the target datasets.

The INSTALL library contains four members which can be used to perform the necessary SMP/E processing:

| | |
|---|---|
| LDSMPA | - Allocates target and DLIB libraries for LD220 |
| LDSMPB | - Allocates SMP system datasets for LD220 (SMPPTS, SMPMTS, etc.) |
| LDSMPC | - Initialise SMP CSI for LD220 |
| LDSMPINS | - Install product using SMP/E (RECEIVE, APPLY, ACCEPT) |

All the jobs need to be tailored to add a valid job card and values for dataset prefix and volume serial etc. as described within the members. The dataset prefix should be the same as the value in the PREFIX JCL variable of the @README job.

After the jobs have been run successfully continue with installation from section as described in section '**Completing Initial Installation'**

# Appendix C – LDAP Schema Updates

LDAP object classes and LDAP attributetypes are added to the schema to provide containers for the licensable TSO command & DATASET management features. These are:

Objectclasses
- racfDataset
- racfPermitId
- ldapxTsoCmd
- racfTsoCmd

Attributetypes
- racfAccess
- racfCreateGroup
- racfDataset
- racfDatasetType
- ldapxTsoOutput
- racfTsoCmdOutput
- ldapxTsoRc
- racfTsoCmdRc
- racfGlobalAudit
- racfPermitId
- racfUnit
- racfVolser
- racfVolsers
- racfWarning
- racfModelDataset
- racfGenericDataset
- racfGroupDataset
- racfEraseDataset
- racfErase
- racfAccessCount
- ldapxInstallationData
- ldapxProgrammerName

For descriptions see their LDIF definitions in member LDIFSCH of the REPRO library.

# Appendix D - Using SSL/TLS to secure connections – possible scenarios

Some companies require IP connections between client and LDAP server to be secured. There is a discussion of the tasks necessary to set up SSL/TLS connectivity to a TDS LDAP server in the 'IBM Tivoli Directory Server Administration and Use for z/OS' manual, and the IBM RACF and IP manuals cover the general topics of using SSL/TLS and certificates in some detail. The following notes describe some different ways this can be done as explored in the set-up of the test environment at One Identity and they are included in case they prove helpful. The RACF commands used are also included. A few further considerations and assumptions are discussed at the end.

Note that in scenario 3 below the use of LDAPX requires additional measures above those needed for just the LDAP task.

**Levels of Authentication**

In most computer installations there will be various operating systems running many different applications. Many of those systems and applications will require some sort of authentication to allow access to the various functions and data they have. Hence any discussion using the word 'authentication' could be open to misinterpretation. To be clear, this discussion is primarily concerned with 'Server authentication' and 'Client authentication' as implemented in the SSL/TLS protocols, which uses digital certificates to identify the session partners. In these scenarios the server is the Tivoli Directory Server (TDS) application, and the values in the TDS start-up parameters (dsconf file) determine whether the SSL/TLS connection uses server authentication or client authentication.

The SSL/TLS Server and Client authentication is about verifying that a valid connection is made and setting up the rules for how the data passing between client and server should be encrypted. It does not (in most cases) determine the scope of access of any specific user/client to functions or data at the server end. So the scenarios below describe how the client signifies which RACF id is going to be used to do the LDAP search. What options are available will depend on the client application being used, but one of the possible ways is by passing the actual userid/password, and in this context that is known as 'Simple authentication', which is distinct/separate from the SSL/TLS server/client authentication. In the third scenario the client is using 'SASL External authentication' to convey the RACF id. . This does blur the distinction between the different authentication methods because information in the client's digital certificate is used to determine the RACF id., but it is still a separate mechanism from the SSL/TLS server/client authentication.

**A bit about certificates**

It is assumed that the reader has some familiarity with the concept of using digital certificates, but here is a very short, simplified, explanation of what it is about. The basic idea is that digital certificates are created and used to identify resources such as systems, applications, servers, users, etc., within an organisation. Those certificates are created (or ***issued*** or ***signed***) by a 'master' (***root***) certificate, which is known as the Certification Authority (CA). Some organisations may create/use their own Certification Authority, others may go to an external Certification Authority that creates/manages certificates for

multiple organisations. The foundation of the idea is that the Certification Authority is considered ***trusted*** within that organisation, so that any resource using a certificate issued / signed by that Certification Authority is recognised as authentic within that organisation. A public copy of the Certification Authority certificate (CA cert) can then be installed on any system that needs to check the integrity of a certificate used by the resource with which it is trying to communicate.

The scenarios below discuss the *use* of certificates and that would be the same regardless of how the Certification Authority is implemented and how the certificates are generated.

**Scenario 1: SSL/TLS Server authentication**

This is the simplest scenario. The client has verification that it is communicating with the expected server. The data between the client and the TDS task is encrypted. Unless the TDS task allows anonymous access the client code will still need to send some authentication such as a RACF id and password in order to retrieve/update data.

| **RACF** | **TDS parms to use/consider** | **Client system** | **Other** |
|---|---|---|---|
| Add CA cert to RACF database (1)<br>Add server certificate to RACF database (2)<br>Define keyring for RACF id of TDS task (3)<br>Add CA certificate to keyring (4)<br>Add server certificate to keyring (4) | listen ldaps://:*portnum*<br>sslKeyRingFile *TDSRACFidkeyring* (3)<br>sslAuth serverAuth<br>sslCertificate *TDSServerCertLabel* (4) | Install CA certificate<br>Select SSL comms (if option is there)<br>Use simple authentication (RACF id and password) to supply BINDUSER | Ensure any firewalls between client and TDS task will allow communication between the chosen IP addresses and port *portnum*<br>Processes to be followed to distribute and install CA certificate on client?<br>Security policy: scope of control of BINDUSER id, i.e. how much access does BINDUSER need<br>Security policy: scope of use of BINDUSER id, e.g. one id per individual, same id used by all, same id used by each distinct functional group, etc. |

**Scenario 2: SSL/TLS Client authentication**

The client has verification that it is communicating with the expected server, and the TDS server end also verifies the certificate of the client trying to connect. The value of the TDS dsconf parameter 'sslAuth' triggers this change of behaviour. The data between the client and the TDS task is encrypted. Note the certificate exchange/verification is done at the TCP/IP level, so it is possible that the client could legitimately connect to the TDS application but then *__not__* have the correct credentials to search/update data. The client code will therefore still need to send some authentication such as a RACF id and password in order to do that.

| RACF | TDS parms to use/consider | Client system | Other |
|---|---|---|---|
| Add CA cert to RACF database (1)<br>• Add server certificate to RACF database (2)<br>• Define keyring for RACF id of TDS task (3)<br>• Add CA certificate to keyring (4)<br>• Add server certificate to keyring (4) | listen ldaps://:*portnum*<br>sslKeyRingFile *TDSRACFidkeyring* (3)<br>sslAuth serverClientAuth<br>sslCertificate *TDSServerCertLabel* (4) | Install CA certificate<br>Install client certificate<br>Select SSL comms (if option is there)<br>Use simple authentication (RACF id and password) to supply BINDUSER | Ensure any firewalls between client and TDS task will allow communication between the chosen IP addresses and port *portnum*<br>Processes to be followed to distribute and install CA certificate and client certificate on client?<br>Security policy: scope of control of BINDUSER id, i.e. how much access does BINDUSER need<br>Security policy: scope of use of BINDUSER id, e.g. one id per individual user, same id used by all, same id used by each distinct functional group, etc.<br>Security policy: scope of client certificates, e.g. one cert per individual user, same one cert used by all, same cert used by each distinct functional group, etc.. |

**Scenario 3: SSL/TLS client authentication, where client certificate defines BINDUSER**

The client has verification that it is communicating with the expected server, and the TDS server end also verifies the certificate of the client trying to connect. The data between the client and the TDS task is encrypted. TDS associates the certificate used by the client with a legitimate RACF id, so the client certificate needs to be added to RACF and connected to the keyring used by the TDS task. The dsconf parameter 'sslMapCertificate' controls exactly how the TDS task should locate the RACF id; the table below shows how it was implemented at One Identity. A further requirement needed for LDAPX is that the Common Name part of the Distinguished Name field in the client certificate needs to be set to the required RACF id too. The client side needs to be told to use 'SASL External' authentication.

| RACF | TDS parms to use/consider | Client system | Other |
|---|---|---|---|
| Add CA cert to RACF database (1)<br>Add server certificate to RACF database (2)<br>Define keyring for RACF id of TDS task (3)<br>Add CA certificate to keyring (4)<br>Add server certificate to keyring (4)<br>Add client certificate(s) to RACF database (5)<br>Add client certificate to keyring (5) | listen ldaps://:*portnum*<br>sslKeyRingFile *TDSRACFidkeyring* (3)<br>sslAuth serverClientAuth<br>sslCertificate *TDSServerCertLabel* (4)<br>sslMapCertificate check fail (5) | Install CA certificate<br>Install client certificate<br>Select SSL comms (if option is there)<br>SASL EXTERNAL authentication | Ensure any firewalls between client and TDS task will allow communication between the chosen IP addresses and port *portnum*<br>Processes to be followed to distribute and install CA certificate and client certificate on client?<br>Security policy: scope of control of BINDUSER id, i.e. how much access does BINDUSER need<br>Security policy: scope of client certificates, e.g. one per individual user, same one used by all, same one used by each distinct functional group, etc..<br>Security policy: each client certificate maps to one RACF id, so does that affect decisions covered in previous bullet point above in terms of using individual certificates or distributing same certificate to many<br>Certificate creation: Common Name (CN) part of Distinguished Name (DN) in certificate needs to have RACF id of required BINDUSER (5) |

**Notes / Examples for above scenarios**

All the RACF commands below can either be issued on TSO interactively or in a batch job. JCL similar to the following was used to issue the RACF commands.

```
//JOBNAME  JOB (ACCT,90),'RACF GENERAL',CLASS=N,MSGCLASS=X
//*
//STEP1 EXEC PGM=IKJEFT01,DYNAMNBR=20
//SYSPRINT DD  SYSOUT=*
//SYSTSPRT DD  SYSOUT=*
//SYSTERM  DD  TERM=TS,SYSOUT=*
//SYSTSIN  DD  *
     RACF commands
/*
```

**(1) Add Certificate Authority certificate to RACF database.**

The CA cert was exported from originating Certification Authority system in BASE64 format and that dataset transported to this system. The label specified in the WITHLABEL operand can be any string which will help to identify this certificate on this system.

```
RACDCERT ADD('fully-qualified-CACERT-BASE64-dataset') -
 CERTAUTH                                             -
 TRUST                                                -
 WITHLABEL('CACERT001-label')
```

**(2) Add server certificate to RACF database.**

The TDS task runs on this mainframe so the RACF database needs to have the private key of the certificate. A suitable certificate could be created completely externally by a Certificate Authority, transported, and added to the RACF database. Alternatively, a certificate could be created in this RACF database and a *certificate signing request* generated and sent to a Certificate Authority to be signed and returned. Examples of both methods are shown below; they are equally valid, but only one method is necessary for any one certificate. In both cases the owner of the certificate is the RACF id used by the TDS started task.

a. Create certificate in RACF
   Use RACDCERT GENCERT command to create the certificate in RACF. The CN, OU, and O values in the certificate distinguished name may need to comply with standards within your organisation.

```
RACDCERT GENCERT                                  -
   ID(TDSracfid)                                  -
   SUBJECTSDN(CN('ThisTDSTask')                   -
   OU('ThisOrgUnit') O('ThisOrg'))                -
   WITHLABEL('TDSServerCertLabel')                -
   NOTAFTER(DATE(2022-12-31))                     -
   SIZE(2048)                                     -
```

```
KEYUSAGE(HANDSHAKE DATAENCRYPT)
```

Use RACDCERT GENREQ command to create a signing request. The request is in Base64 format which is recognisable/usable by any Certificate Authority system. It is written to a dataset, which could then be sent using FTP or as an e-mail attachment to the Certificate Authority system. As the Base64 format uses only printable characters it is also possible to copy the data directly from the dataset (using cut/paste) into a file/dataset on the Certificate Authority system if the invoker has the necessary authority.

```
RACDCERT GENREQ(LABEL('TDSServerCertLabel'))          -
     ID(TDSracfid)                                     -
     DSN('Fully-Qualified-DSN-For-Signing-Request')
```

The Certificate Authority uses data from the certificate request and 'signs' it using the necessary Root certificate(s). The signed certificate is exported into a dataset in Base64 format so it can be transported back to this system. After creating a dataset on this system containing the exported data, the signed portion of the certificate is then added to this RACF database, associating it with the originally created certificate by specifying the same label. The following RACDCERT ADD command illustrates this.

```
RACDCERT ADD('Fully-Qualified-Base64-DSN-With-CA-Signed-TDS-Server-Cert') -
  ID(TDSracfid)                                                           -
  TRUST                                                                   -
  WITHLABEL('TDSServerCertLabel')
```

b. Create certificate externally
The necessary information to create certificate has to be provided to the Certificate Authority, including expiry date, the Distinguished Name (DN) values (i.e. Common name (CN=), Organisation (O=), etc.), and key length. A Certificate Authority, or some intermediate agent, may provide a form to prompt the requester for the information. The signed certificate created by the Certificate Authority contains the private key so in order to transport it back safely to the requester it is exported into a dataset in a format known as a 'PKCS12' package which is protected with a simple password which is supplied to the requester separately. The password is needed when RACF installs the certificate as shown below.

```
RACDCERT ADD('Fully-Qualified-PKCS12-DSN-With-CA-Signed-TDS-Server-Cert') -
    ID(TDSRACFid)                                                        -
    TRUST                                                                -
    PASSWORD('PKCS12pass')                                                 -
    WITHLABEL('TDSServerCertLabel')
```

**(3)** Define keyring for RACF id of TDS started task. This needs to be specified in the TDS parms (dsconf) file.

```
RACDCERT ADDRING(TDSRACFidkeyring) -
  ID(TDSracfid)
```

So in dsconf file: sslKeyRingFile *TDSRACFidkeyring*

**(4)** Add CA cert and server cert to keyring. Need to tell TDS task which certificate it uses by setting value in dsconf file

```
RACDCERT ID(TDSracfid)    -
     CONNECT(              -
     CERTAUTH              -
     RING(TDSRACFidkeyring)          -
    LABEL('CACERT001-label'))

  RACDCERT ID(TDSracfid)    -
     CONNECT(              -
     ID(TDSracfid)         -
     RING(TDSRACFidkeyring)          -
     DEFAULT               -
    LABEL('TDSServerCertLabel'))
```

So in dsconf file: sslCertificate *TDSServerCertLabel*

**(5)** Add client certificate(s) to RACF database. Add client certificate to keyring.

Similar to the discussion under Note 2b above about obtaining a server certificate, the client certificate was wholly created on the Certification Authority system. The whole certificate with private key was exported from Certificate Authority system in PKCS12 format protected with a password. In scenario 2 the certificate was installed just on the client system – that process is not discussed here. In scenario 3 it also has to be added to the RACF database and keyring. For LDAPX to pick up the BINDUSER correctly a Common Name (CN) value in the Distinguished Name (DN) of the certificate needs to be set to the RACF id that owns the client certificate. The RACF commands to add the client cert to RACF and then add it to a keyring are shown below.

```
RACDCERT ADD('Fully-Qualified-PKCS12-DSN-With-CA-Signed-Client-Cert') -
    ID(ClientRACFid)                -
    TRUST                           -
    PASSWORD('PKCS12pass')            -
    WITHLABEL('Client1-Cert-Label')

 RACDCERT ID(TDSracfid)             -
    CONNECT(                        -
    ID(ClientRACFid)                -
    RING(TDSRACFidkeyring)                    -
   LABEL('Client1-Cert-Label'))
```

The distinguished name of the client certificate in this case is:
CN=*ClientRACFid*,OU=ThisOrgUnit,O=Org

As an additional note, the Certificate Authority used for One Identity z/OS testing environment runs on a separate z/OS system, so RACF is used to manage certificates on there too. So the client certificate mentioned above was created using a command like the following:

```
RACDCERT SITE                                        -
```

```
GENCERT                                          -
SUBJECTSDN(CN('ClientRACFid')                    -
OU('ThisOrgUnit') O('Org'))                      -
WITHLABEL ('Client1-Cert-Label')                 -
NOTAFTER(DATE(2022-12-31))                       -
SIZE(2048)                                       -
SIGNWITH(CERTAUTH LABEL('CACERT001-label'))      -
KEYUSAGE(HANDSHAKE DATAENCRYPT)
```

**Further Considerations**

A userid with 'System Special' authority was used to issue the above commands, although it is possible to use a non system special id if access to the necessary RACF commands has been authorised. See RACF manuals for details of what level of authority is required to issue particular RACF commands.

The format of some of the above commands will change slightly if an ISCF PKDS is being used to store private keys. Again, best to refer to RACF manuals for details.

## Appendix E – the @README install job

A copy is listed here for review before starting the install. This job will be in member @README of the …BINPDS library when step 5 of the unzip & TRANSMIT procedure detailed in README.TXT has been performed. It is intended to be run in 2 or 3 sections (1st starting at the ALLOC step to create 5 new PDS libraries, 2nd starting at CMDEDIT to prepare sample configuration & JCL, then optionally 3rd at COPYIN). You would omit the last restart point if you prefer to edit the 'live' TDS configuration manually via ISPF, using the configuration files prepared by steps TSOOMVS & USSEDIT as reference rather than running the script created by step COPYIN to copy them to the live directory.

```
//+++++++I JOB (0,ACCT),'UNTRANSMIT LD220E',CLASS=N,
//             RESTART=PART1,    easier restart label for part1
//*             RESTART=ALLOC,     SKIP THE BINPDS STEPS (USUAL)
//*             RESTART=CMDEDIT,  RESTART HERE IF USED SMP INSTALL
//*             RESTART=PART2,    easier restart label for part2
//*             RESTART=COPYIN,   SKIP TO COPYING FILES FROM OMVS
//*             RESTART=PART3,    easier restart label for part3
//*             RESTART=TSOOMVS,   SKIP TO COPYING FILES TO OMVS
//*             RESTART=LDINST,   SKIP TO COPYING FILES TO OMVS
//             MSGCLASS=X,NOTIFY=&SYSUID
//*********************************************************************
//*
//* This job reconstitutes LDAPX from a flat file using TSO receive.
//* Please review all of this comment block before proceeding.
//*
//* Please review the values on the 1st block of SET statements before
//* submitting this job for the 1st time.
//*
//* The file LD220E.BIN contains the LDAPX (level LD220E)
//* Program Product libraries required for execution on an IBM z
//* mainframe.  The file should be copied to a mainframe using binary
//* file transfer, then JCL similar to that below can be used to
//* reconstitute the separate mainframe libraries, which were
//* originally packed into the one sequential file by first using the
//* 'TSO transmit' command on each library individually, then those
//* sequential output files copied into members of a pds, which was
//* then itself the subject of another 'TSO transmit' command.
//*
//* Note: this job as supplied skips the steps which create file BINPDS
//* because the first time you see this you must have already created
//* it using the instructions in the README.TXT pc file. They are
//* included for completeness and so the whole job can be stored
//* elsewhere for reruns.  Also see comments re SMP if intend to use.
//*
//*
//* TO RUN THIS JOB (in 3 parts) DO THE FOLLOWING
//*
//*    part1:
//*    a) review & edit the values in the 1st block of SET statements
//*       and change all occurances of /usr/oneiden to a directory you
//*       have permissions to update and execute and TDS will have.
//*    b) review & check the JCL below to see that it will create &
//*       fill new libraries and then stop after step EDIT2.
//*    c) submit this member and check libraries are created.
//*
//*       If you need to rerun part1 first delete the files it created.
//*       Do not try to run part1 & part2 as one job, because the newly
```

```
//*       created pds files will not be shareable for editing.
//*
//*
//*    If SMP is intended, set PRESMP below so the files created by
//*    part1 have relfile suffixes and run the 4 smp jobs before part2,
//*    because the files part1 creates will be treated as SMP relfiles,
//*    and the actual target libraries will be created by the smp apply.
//*    If SMP is not being used, leave the PRESMP value as null.
//*
//*
//*    part2:
//*    d) review & check the steps CMDEDIT to EDIT4 will customize
//*       members of the libraries created above and then stop.
//*    e) comment out the // null stmt before step PART2
//*    f) alter RESTART stmts in jobcard to restart at PART2 or CMDEDIT
//*    g) submit this member and check customization has been performed.
//*
//*       Part2 can be rerun if you need to alter customizations.
//*
//*
//*    part3 (if applicable):
//*    h) if TDS is not already configured or running and it is ok
//*       to create or replace TDS config files ds.conf & ds.envvars
//*       then review & restart this job again at step PART3 or COPYIN
//*
//*       Part2 and part3 can be rerun together if neccessary.
//*
//*
//*    manual alternative (instead of part3):
//*    i) if TDS was already configured or running then review the
//*       script prepared in ...TEMP(TSOOMVS) and manually compare and
//*       edit the changes to ds.conf, ds.envvars, & proclib that the
//*       script would copy. Or the relevant output definitions in
//*       part3 could be customised to create new files, ds-nnn.conf
//*       etc, so part3 could be run without it changing existing files.
//*       (see the proclib member for how to use a ds.conf alternative)
//*
//*
//* AFTER RUNNING THIS JOB DO FOLLOWING TO COMPLETE THE INSTALL:
//*
//* After completing the above steps, continue by reviewing,
//* possibly customizing, and then running 2 jobs, as supplied in:
//*    1) ...CNTL(LDAPRACF) to issue RACF commands required for TDS
//*       (not all commands will be required if TDS is already set up)
//* then start the LDAP server via MVS command
//*       S DSSRV,PARMS='-D PLUG' (optionally plus other debug parms)
//* then run the 2nd job
//*    2) ...CNTL(LDAPMOD)  to extend the TDS schema to cover ldapx
//*
//* NOTE: The 2 jobs above setup and then supply a password
//* of 'notthis' for a new RACF userid LDAPADM intended as a sample
//* of a SPECIAL user required in order to bind to LDAP and send
//* requests to SDBM & the ldapx plugin.  You should change the
//* password of LDAPADM once the installation is complete, and use
//* the new password to bind to LDAP when making requests,
//* or alternatively you may use any pre-existing SPECIAL user for
//* the bind provided they have a TSO segment and OMVS(UID(0)).
//*
//* Another, optional, job is also provided but strictly as an asis
//* sample, which MAY help if SSL is required for 'LDAP over SSL':
```

```
//*    3) ...CNTL(SAMPSSL)  sample RACDCERT commands that may help
//* For this, and all other aspects of configuring LDAP & SDBM,
//* the definitive reference is a manual with the following title
//*    "IBM Tivoli Directory Server Administration and Use for z/OS"
//* (there are versions for each z/OS level).
//*
//* Also provided is an asis sample that may be adapted for testing
//* LDAP requests:
//*    3) ...CNTL(LDAPTEST)  sample batch job to run IBM LDAP client
//* tools ldapsearch and/or ldapmodify to send LDAP requests to TDS
//* and show resultant entries or messages.
//*
//*
//* Some basic information from TDS manuals:
//*    to start TDS without trace messages from the plugin:
//*       S DSSRV
//*    to start TDS with trace messages from the plugin:
//*       S DSSRV,PARMS='-D PLUGIN'
//*    to start TDS with more trace messages, from plugin & TDS:
//*       S DSSRV,PARM='-D BER+PLUG+TRAC+THRE+SDBM'
//*    to alter tracing dynamically:
//*       F DSSRV,DEBUG PLUG+SDBM+BER+PACK+FILT' etc
//*       F DSSRV,DEBUG OFF'
//*    to stop TDS:
//*       P DSSRV
//*
//*    client bind user needs: SPECIAL, or many site-specific
//*       permissions if not SPECIAL as per TDS/RACF manuals,
//*       depending on what profiles are to be  administered and on
//*       the hierarchy of ownership & RACF admin scope.
//*    TDS task user needs: SPECIAL + a TSO segment + OMVS(UID(0))
//*       or site-specific permissions if not as above.
//*       NOPASSWORD is recommended, to prevent its use online,
//*       since this user is for an STC only and not for 3270s or jobs.
//*       (the TSO segment is required for updates by the ldapx plugin)
//*
//*    configuration file: /etc/ldap/ds.conf
//*       the ldapx extension is included by stmts starting 'plugin'.
//*    schema file:        /var/ldap/schema/schema.db
//*
//*    schema.db can be deleted, server will recreate default schema
//*    for SDBM when next restarted. CNTL(LDAPMOD) will need rerunning
//*
//*******************************************************************
//         EXPORT SYMLIST=(PREFIX,USSDIR)
//*******************************************************************
//* ALTER THE VALUES ON THESE SET STATEMENTS BEFORE JOB SUBMISSION:
//*******************************************************************
//         SET PREFIX='CANDKQ.LD220E' DSN PREFIX FOR NEW LIBRARIES
//         SET USSDIR='/usr/oneiden'  EXISTING USS DIR
//         SET PRESMP=''              NO EXTRA PREFIX FOR NON-SMP
//*         SET PRESMP='.RELFILE'      EXTRA PREFIX REQUIRED FOR SMP
//* NOTE: if using SMP then run the 4 smp jobs between part1 & part2
//         SET DISKVOL=XXXXXX         DISK VOLUME FOR NEW LIBRARIES
//         SET PROCLIB='XXXX.PROCLIB' DSN OF PROCLIB FOR DSSRV JCL
//         SET STEPLIB='XXXX.LINKLIB' DSN OF STEPLIB FOR PLUGIN
//         SET    CLIB=XXXX.CLIST     DSN of SYSPROC for tsocmd script
//         SET SYSEXEC=XXXX.REXX      DSN of SYSEXEC for tsocmd script
//         SET ISPLLIB=XXXX.LINKLIB   DSN of ISPLLIB for tsocmd script
//         SET SYSUADS=XXXX.UADS      DSN of SYSUADS for tsocmd script
```

```
//*
//*******************************************************************
//*
//* ADVANCED SET OPTIONS - YOU SHOULD NOT NEED TO ALTER THESE:
//*
//         SET VOLSER=&DISKVOL        DISK VOLUME FOR NEW LIBRARIES
//         SET DISK=SYSALLDA          UNIT TYPE FOR DISK
//         SET DUNIT=SYSALLDA         UNIT TYPE FOR DISK
//         SET CNTLBLK='9040'      BLOCKSIZE FOR NEW CONTROL LIBRARY
//         SET LOADBLK='6144'      BLOCKSIZE FOR NEW LOAD    LIBRARY
//         SET PROFBLK='9000'      BLOCKSIZE FOR NEW PROFILE LIBRARY
//         SET SAMPBLK='9040'      BLOCKSIZE FOR NEW SOURCE  LIBRARY
//         SET PANLBLK='6144'      BLOCKSIZE FOR NEW PANELS  LIBRARY
//         SET PBLIBLK='8976'      BLOCKSIZE FOR NEW PBLIB   LIBRARY
//*        SET DATABLK='9076'      BLOCKSIZE FOR NEW REPRO   LIBRARY
//         SET EDITBLK='3120'      BLOCKSIZE FOR NEW REPRO EDIT ENABLED
//         SET HELPBLK='6336'      BLOCKSIZE FOR NEW HELP    LIBRARY
//         SET MSGBLK='6300'       BLOCKSIZE FOR NEW MESSAGE LIBRARY
//*
//*******************************************************************
//*
//ALLOC1  EXEC PGM=IEFBR14         DEFAULT JOB CARD BYPASSES THIS STEP
//BINPDS    DD DISP=(,CATLG),DSN=&PREFIX..BINPDS,
//             UNIT=&DISK,VOL=SER=&DISKVOL,
//             SPACE=(3120,(1000,1000,10),RLSE),
//             DCB=(RECFM=FB,LRECL=80,BLKSIZE=3120)
//*
//TSO1    EXEC PGM=IKJEFT01,DYNAMNBR=40
//SYSPRINT  DD TERM=TS,SYSOUT=*,HOLD=YES
//SYSTERM   DD TERM=TS,SYSOUT=*,HOLD=YES
//SYSTSPRT  DD TERM=TS,SYSOUT=*,HOLD=YES   /* TERMINAL OUTPUT
//SYSTSIN   DD *
RECEIVE INDATASET('YOURPREFIXES.BIN')  /* ONLY NEED TO ALTER THIS
 DSNAME('YOURPREFIXES.BINPDS') OLD     /* IF YOU ALTERED RESTART=
//*
//*******************************************************************
//* THIS JOB IS DESIGNED TO HAVE NO MANUAL ALTERATIONS BELOW HERE
//*******************************************************************
//PART1   EXEC PGM=IEFBR14         easier to remember restart label
//*
//*   ALLOCATE INSTALLATION JCL LIBRARIES THAT ARE ALWAYS OUTSIDE SMP
//*
//ALLOC   EXEC PGM=IEFBR14         DEFAULT JOB STARTS RUNNING HERE
//TEMP      DD DSN=&PREFIX..TEMP,
//             DISP=(,CATLG),
//             UNIT=&DISK,VOL=SER=&DISKVOL,
//             SPACE=(9040,(10,10,10),RLSE),
//             DCB=(RECFM=FB,LRECL=80,BLKSIZE=9040)
//INSTALL   DD DSN=&PREFIX..INSTALL,
//             DISP=(,CATLG),
//             UNIT=&DISK,VOL=SER=&DISKVOL,
//             SPACE=(9040,(10,10,10),RLSE),
//             DCB=(RECFM=FB,LRECL=80,BLKSIZE=9040)
//*
//ALLOC2  PROC PREFIX=&PREFIX.&PRESMP
//ALLOC2  EXEC PGM=IEFBR14
//CNTL      DD DSN=&PREFIX..CNTL,
//           UNIT=&DISK,
//           DISP=(,CATLG),
//           DCB=(BLKSIZE=&CNTLBLK,LRECL=80,RECFM=FB),
```

```
//           SPACE=(6320,(44,114,7),RLSE),
//           VOL=SER=&DISKVOL.
//LOAD      DD DSN=&PREFIX..LOAD,
//           UNIT=&DISK,
//           DISP=(,CATLG),
//           DCB=(BLKSIZE=&LOADBLK,RECFM=U),
//           SPACE=(6144,(2332,233,105),RLSE),
//           VOL=SER=&DISKVOL.
//REPRO     DD DSN=&PREFIX..REPRO,
//           UNIT=&DISK,
//           DISP=(,CATLG),
//           DCB=(BLKSIZE=&EDITBLK,LRECL=255,RECFM=VB), max tso edit
//           SPACE=(9076,(264,26,6),RLSE),
//           VOL=SER=&DISKVOL.
//FIX       DD DSN=&PREFIX..FIXLIST,
//           UNIT=&DISK,
//           DISP=(,CATLG),
//           DCB=(BLKSIZE=&CNTLBLK,LRECL=80,RECFM=FB),
//           SPACE=(9040,(1145,114,193),RLSE),
//           VOL=SER=&DISKVOL.
//JCLIN     DD DSN=&PREFIX..JCLIN,
//             DISP=(,CATLG),
//             UNIT=&DISK,VOL=SER=&DISKVOL,
//             SPACE=(9040,(10,10,10),RLSE),
//             DCB=(RECFM=FB,LRECL=80,BLKSIZE=9040)
//        PEND
//ALLREL  EXEC ALLOC2         ALLOCATE POTENTIAL RELFILES
//*
//*
//*   COPY SOME TSO COMMANDS INTO TEMP(TSOCMDS)
//*
//TSOCMDS EXEC PGM=IEBGENER
//SYSPRINT  DD SYSOUT=*
//SYSUT1    DD *
 /*************************************************************/
 /* THIS IS A LIST OF COMMANDS TO PERFORM TSO RECEIVE FUNCTION */
 /* NOTE: DO NOT CHANGE -PREF- MANUALLY - IT IS DONE BY NEXT STEP
 /*************************************************************/
 /* THE FOLLOWING FILE IS ALREADY RESTORED:
 /*   RECEIVE INDATASET('--PREF---.BIN')
 /*    DSNAME('---PREF---.BINPDS') OLD
 /* THE REST WILL BE RESTORED FROM THE MEMBERS OF ...BINPDS:
 /*************************************************************/
RECEIVE INDATASET('--PREF--.BINPDS(INSTALL)')
 DSNAME('--PREF--.INSTALL') OLD
RECEIVE INDATASET('--PREF--.BINPDS(CNTL)')
 DSNAME('--PREF----SMP--.CNTL') OLD
RECEIVE INDATASET('--PREF--.BINPDS(FIXLIST)')
 DSNAME('--PREF----SMP--.FIXLIST') OLD
 /* RECEIVE INDATASET('--PREF--.BINPDS(HELP)')
 /*  DSNAME('--PREF----SMP--.HELP') OLD
RECEIVE INDATASET('--PREF--.BINPDS(LOAD)')
 DSNAME('--PREF----SMP--.LOAD') OLD
 /* RECEIVE INDATASET('--PREF--.BINPDS(MACLIB)')
 /*  DSNAME('--PREF----SMP--.MACLIB') OLD
 /* RECEIVE INDATASET('--PREF--.BINPDS(MESSAGE)')
 /*  DSNAME('--PREF----SMP--.MESSAGE') OLD
 /* RECEIVE INDATASET('--PREF--.BINPDS(PANELS)')
 /*  DSNAME('--PREF----SMP--.PANELS') OLD
 /* RECEIVE INDATASET('--PREF--.BINPDS(PBLIB)')
```

```
 /*  DSNAME('--PREF----SMP--.PBLIB') OLD
RECEIVE INDATASET('--PREF--.BINPDS(REPRO)')
 DSNAME('--PREF----SMP--.REPRO') OLD
 /* RECEIVE INDATASET('--PREF--.BINPDS(SOURCE)')
 /*  DSNAME('--PREF----SMP--.SOURCE') OLD
 /* note: if you indicated a non-SMP install, by not setting the extra
 /* prefix for SMP in symbol PRESMP within the BINPDS(@README) job,
 /* then the remaining TSO commands, below, will have been
 /* commented out by step EDIT of that job in order to avoid
 /* setting up files and aliases only required for an SMP install.
   /* populate the jclin relfile by copying 1 member from install */
REPRO INDATASET('--PREF--.INSTALL(ZZLD220)') +
      OUTDATASET('--PREF----SMP--.JCLIN(ZZLD220)')
   /* define aliases so can receive the above files as smp relfiles */
   /* (added 8nov06 for ability to do smp install from the download) */
DEFINE ALIAS (NAME('--PREF--.RELFILE.ZZLD220.F1') +
 RELATE('--PREF----SMP--.JCLIN')
DEFINE ALIAS (NAME('--PREF--.RELFILE.ZZLD220.F2') +
 RELATE('--PREF----SMP--.LOAD')
 /* DEFINE ALIAS (NAME('--PREF--.RELFILE.ZZLD220.F3') +
 RELATE('--PREF----SMP--.PANELS')
 /* DEFINE ALIAS (NAME('--PREF--.RELFILE.ZZLD220.F4') +
 RELATE('--PREF----SMP--.PBLIB')
 /* DEFINE ALIAS (NAME('--PREF--.RELFILE.ZZLD220.F5') +
 RELATE('--PREF----SMP--.SOURCE')
 /* DEFINE ALIAS (NAME('--PREF--.RELFILE.ZZLD220.F6') +
 RELATE('--PREF----SMP--.MACLIB')
DEFINE ALIAS (NAME('--PREF--.RELFILE.ZZLD220.F7') +
 RELATE('--PREF----SMP--.FIXLIST')
DEFINE ALIAS (NAME('--PREF--.RELFILE.ZZLD220.F3') +
 RELATE('--PREF----SMP--.CNTL')
DEFINE ALIAS (NAME('--PREF--.RELFILE.ZZLD220.F4') +
 RELATE('--PREF----SMP--.REPRO')
 /* DEFINE ALIAS (NAME('--PREF--.RELFILE.ZZLD220.F10') +
 RELATE('--PREF----SMP--.HELP')
 /* DEFINE ALIAS (NAME('--PREF--.RELFILE.ZZLD220.F11') +
 RELATE('--PREF----SMP--.MESSAGE')
//SYSUT2    DD DSN=&PREFIX..TEMP(TSOCMDS),DISP=SHR
//SYSIN     DD DUMMY
//*
//*   CREATE A CLIST THAT WILL SET THE DATASETNAMES IN TEMP(TSOCMDS)
//*
//TSOEDIT EXEC PGM=IEBGENER
//SYSPRINT  DD SYSOUT=*
//SYSUT1    DD *
PROC 2 PREFIX PRESMP
CONTROL LIST SYMLIST CONLIST NOFLUSH END(ENDD)
ERROR DO       /* keep going eg after a 'text not found' msg
   SET &POS = HANDLED       /* indicate error occurred
   SET &SAVECC = &LASTCC    /* save the error code
   RETURN                   /* ...to next stmt
ENDD
 /* this clist changes the commands to use &prefix from jcl
 /* a leading dot will have been added to &presmp so remove it first
IF &PRESMP = . THEN SET &PRESMP =
ELSE IF &SUBSTR(1:2,&PRESMP  ) = .. THEN -
   SET &PRESMP = &SUBSTR(2:&LENGTH(&STR(&PRESMP)),&PRESMP)
E '&PREFIX..TEMP(TSOCMDS)' DATA
V ON
L
```

```
REN 1 1
CHANGE 1 9999 /--PREF--/&PREFIX/ ALL
CHANGE 1 9999 /--SMP--/&PRESMP/ ALL
IF &PRESMP = THEN CHANGE 1 9999 +DEFINE ALIAS+/*DEFINE ALIAS+ ALL
IF &PRESMP = THEN CHANGE 1 9999 +REPRO INDA+/*REPRO INDA+ ALL
L
END SAVE
IF &PRESMP = THEN -
   DEL '&PREFIX..JCLIN'   /* not needed for non-smp
EXIT
//SYSUT2    DD DSN=&PREFIX..TEMP(TSOEDIT),DISP=SHR
//SYSIN     DD DUMMY
//*
//*   RUN THE EDIT OF TEMP(TSOCMDS)
//*
//EDIT    EXEC PGM=IKJEFT01,DYNAMNBR=40,   EDIT THE TSO RECEIVE CMDS
//             PARM='%TSOEDIT &PREFIX .&PRESMP'
//SYSPROC   DD DSN=&PREFIX..TEMP,DISP=SHR
//SYSTSPRT  DD TERM=TS,SYSOUT=*,HOLD=YES   /* TERMINAL OUTPUT
//SYSTSIN   DD DUMMY
//*
//*   RUN THE TSO RECEIVE COMMANDS ETC FROM TEMP(TSOCMDS)
//*
//RECV    EXEC PGM=IKJEFT01,DYNAMNBR=40,   RUN THE TSO RECEIVE CMDS
//             PARM=''                     PLUS SOME DEFINE ALIASES
//SYSPRINT  DD TERM=TS,SYSOUT=*,HOLD=YES
//SYSTSPRT  DD TERM=TS,SYSOUT=*,HOLD=YES   /* TERMINAL OUTPUT
//SYSUT3    DD UNIT=VIO,SPACE=(CYL,(5,5))  /* FOR IEBCOPY SPILL
//SYSUT4    DD UNIT=VIO,SPACE=(CYL,(5,5))  /* FOR IEBCOPY SPILL
//SYSTSIN   DD DSN=&PREFIX..TEMP(TSOCMDS),DISP=SHR
//*
//*
//*   CREATE CLIST THAT CAN ALTER THE SMP JCL TO USE DISK INPUT NOT TAPE
//*
//INSEDIT EXEC PGM=IEBGENER
//SYSPRINT  DD SYSOUT=*
//SYSUT1    DD *
PROC 1 PREFIX
CONTROL LIST SYMLIST CONLIST NOFLUSH END(ENDD)
ERROR DO       /* keep going eg after a 'text not found' msg
   SET &POS = HANDLED       /* indicate error occurred
   SET &SAVECC = &LASTCC    /* save the error code
   RETURN                   /* ...to next stmt
ENDD
 /* this clist adds the disk rfprefix into the smp receive job
 /* and changes the smpmcs input (thus relfiles too) from tape to disk
 /* and changes standard install/maint jobs to use the restored files
SET &INSTLIB = &PREFIX..INSTALL   /* or &PREFIX..CNTL
E '&INSTLIB(LDSMPINS)' DATA
V ON
L
REN 1 1
CHANGE 1 9999 /SYSMODS ./SYSMODS RFPREFIX(&PREFIX..RELFILE) ./
CHANGE 1 9999 /SYSMODS RFPREFIX(--PREF--.RELFILE) .+
              /SYSMODS RFPREFIX(&PREFIX..RELFILE) ./
CHANGE 1 9999 .*SMPPTFIN  DD DSN=*PREFIX*.SMPPTFIN  DD DSN=*PREFIX*.
CHANGE 1 9999 .SMPPTFIN  DD DSN=SMPMCS.*SMPPTFIN  DD DSN=SMPMCS.
CHANGE 1 9999 .            DISP=.*            DISP=.
CHANGE 1 9999 .            VOL=.*            VOL=.
L
```

```
END SAVE
E '&INSTLIB(LDAPINST)' DATA
V ON
L
REN 1 1
CHANGE 1 9999 /IEFBR14=Y/IEFBR14=N/
CHANGE 1 9999 /CPYPASS=Y/CPYPASS=N/
L
END SAVE
EXIT
//SYSUT2    DD DSN=&PREFIX..TEMP(TSOEDIT2),DISP=SHR
//SYSIN     DD DUMMY
//*
//*   RUN THE EDIT OF SMP INSTALL JCL IN INSTALL(LDSMPINS)
//*
//EDIT2   EXEC PGM=IKJEFT01,DYNAMNBR=40,   EDIT ...INSTALL(SESMPINS)
//             PARM='%TSOEDIT2 &PREFIX'
//SYSPROC   DD DSN=&PREFIX..TEMP,DISP=SHR
//SYSTSPRT  DD TERM=TS,SYSOUT=*,HOLD=YES   /* TERMINAL OUTPUT
//SYSTSIN   DD DUMMY
//*
//*   If installing with SMP then stop the 1st run of this job here
//*   by removing the '**' from the line below (so it just has // ).
//*   Run the 4 smp jobs smpa/b/c/ins then put the '**' back and
//*   restart this job at the next step ie RESTART=CMDEDIT
//**
//*================== 1st stop here before continuing =================
//
//*========= restart here after commenting out the null above =========
//PART2   EXEC PGM=IEFBR14         easier to remember restart label
//**
//*   CREATE A CLIST THAT CAN CUSTOMIZE THE SUPPLIED REPRO(SCRIPTSO)
//*
//CMDEDIT EXEC PGM=IEBGENER
//SYSPRINT  DD SYSOUT=*
//SYSUT1    DD *
PROC 5 PREFIX CLIB SYSEXEC ISPLLIB SYSUADS
CONTROL LIST SYMLIST CONLIST NOFLUSH END(ENDD) ASIS
ERROR DO       /* keep going eg after a 'text not found' msg
   SET &POS = HANDLED       /* indicate error occurred
   SET &SAVECC = &LASTCC    /* save the error code
   RETURN                   /* ...to next stmt
ENDD
WRITE export SYSPROC=&CLIB
WRITE export SYSEXEC=&SYSEXEC
WRITE export ISPLLIB=&ISPLLIB
WRITE export SYSUADS=&SYSUADS
E '&PREFIX..REPRO(SCRIPTSO)' TEXT ASIS
V ON
L
REN 10 10
TOP
find '#!' 1
in #
in # This was customized in REPRO(SCRIPTSO) before copying to USS
in #
find 'export SYSPROC=' 1
change * 'export' '#export'
in export SYSPROC=&CLIB
find 'export SYSEXEC=' 1
```

```
change * 'export' '#export'
in export SYSEXEC=&SYSEXEC
find 'export ISPLLIB=' 1
change * 'export' '#export'
in export ISPLLIB=&ISPLLIB
find 'export SYSUADS=' 1
change * 'export' '#export'
in export SYSUADS=&SYSUADS
L
UNNUM
L
END SAVE
E '&PREFIX..CNTL(LDAPXCFG)' TEXT ASIS
V ON
L
REN 10 10
TOP
IF (&CLIB NE 'XXXX.CLIST' AND &CLIB NE XXXX.CLIST)THEN -
 DO
  find 'tsocmdSYSPROC=' 1
  change * 'tsocmdSYSPROC=' ' tsocmdSYSPROC='
  in tsocmdSYSPROC=&CLIB
 ENDD
IF (&SYSEXEC NE 'XXXX.REXX' AND &SYSEXEC NE XXXX.REXX)THEN -
 DO
  find 'tsocmdSYSEXEC=' 1
  change * 'tsocmdSYSEXEC=' ' tsocmdSYSEXEC='
  in tsocmdSYSEXEC=&SYSEXEC
 ENDD
IF (&ISPLLIB NE 'XXXX.LINKLIB' AND &ISPLLIB NE XXXX.LINKLIB)THEN -
 DO
  find 'tsocmdISPLLIB=' 1
  change * 'tsocmdISPLLIB=' ' tsocmdISPLLIB='
  in tsocmdISPLLIB=&ISPLLIB
 ENDD
IF (&SYSUADS NE 'XXXX.UADS' AND &SYSUADS NE XXXX.UADS)THEN -
 DO
  find 'tsocmdSYSUADS=' 1
  change * 'tsocmdSYSUADS=' ' tsocmdSYSUADS='
  in tsocmdSYSUADS=&SYSUADS
 ENDD
L
UNNUM
L
END SAVE
EXIT
//SYSUT2    DD DSN=&PREFIX..TEMP(TSOEDIT3),DISP=SHR
//SYSIN     DD DUMMY
//*
//*   RUN THE EDIT OF THE INSTALL SCRIPT FOR OMVS
//*
//EDIT3   EXEC PGM=IKJEFT01,DYNAMNBR=40,   EDIT ...TEMP(TSOOMVS)
//*       PARM=('%TSOEDIT3 &PREFIX &CLIB',
//*            ' &SYSEXEC &ISPLLIB &SYSUADS')
// PARM='%TSOEDIT3 &PREFIX &CLIB &SYSEXEC &ISPLLIB &SYSUADS'
//SYSPROC   DD DSN=&PREFIX..TEMP,DISP=SHR
//SYSTSPRT  DD TERM=TS,SYSOUT=*,HOLD=YES   /* TERMINAL OUTPUT
//SYSTSIN   DD DUMMY
//*
//*
```

```
//*   COPY IBM SAMPLE DSSRV, ds.conf & ds.envvars SO CAN EDIT THEM
//*
//COPYDS  EXEC PGM=IKJEFT01
//DSIN1     DD PATH='/usr/lpp/ldap/etc/ds.conf',
//             PATHOPTS=(ORDONLY)
//DSIN2     DD PATH='/usr/lpp/ldap/etc/ds.envvars',
//             PATHOPTS=(ORDONLY)
//DSIN3     DD DSN=GLD.SGLDSAMP(DSSRV),DISP=SHR
//DSOUT1    DD DSN=&PREFIX..CNTL(DSCONF),DISP=SHR
//DSOUT2    DD DSN=&PREFIX..CNTL(ENVVARS),DISP=SHR
//DSOUT3    DD DSN=&PREFIX..CNTL(DSSRV),DISP=SHR
//SYSTSPRT  DD SYSOUT=*
//SYSTSIN   DD *
OCOPY INDD(DSIN1) OUTDD(DSOUT1) TEXT CONVERT(YES) PATHOPTS(USE)
OCOPY INDD(DSIN2) OUTDD(DSOUT2) TEXT CONVERT(YES) PATHOPTS(USE)
OCOPY INDD(DSIN3) OUTDD(DSOUT3)
//*
//*   CREATE A SCRIPT THAT CAN COPY COMPONENTS TO OMVS
//*
//TSOOMVS EXEC PGM=IEBGENER
//SYSPRINT  DD SYSOUT=*
//SYSUT1    DD *,SYMBOLS=JCLONLY
#*****************************************************************
#* THIS IS A LIST OF OMVS COMMANDS TO PERFORM DURING THE INSTALL *
#* NOTE: DO NOT CHANGE -PREF- MANUALLY - IT IS DONE BY NEXT STEP *
#*****************************************************************
# turn on listing each command
set -x
#cd /usr
#mkdir oneiden
uss='&USSDIR'
cd $uss
#* requires TDS user to have a TSO segment else tsocmd will not work
cp -T "//'--PREF----SMP--.REPRO(SCRIPTSO)'" $uss/scriptso
chmod 775 scriptso
#* an alternative with echo statements to show it was at least invoked
cp -T "//'--PREF----SMP--.REPRO(SCRIPTSE)'" $uss/scriptsoecho
chmod 775 scriptsoecho
cp -T "//'--PREF----SMP--.REPRO(LDIFSCH)'" $uss/ldifschema.txt
#   see https://www-01.ibm.com/support/knowledgecenter/#!/
#             SSLTBW_1.13.0/com.ibm.zos.r13.e0zm100/shelluti.htm
#   (tsocmd used to be an optional ibm tools download but is std now)
# cp -T "//'--PREF----SMP--.REPRO(TSOCMD)'" $uss/tsocmd
cp -T "//'--PREF----SMP--.CNTL(DSCONF)'" $uss/ds.conf
cp -T "//'--PREF----SMP--.CNTL(ENVVARS)'" $uss/ds.envvars
ls -alHi
//SYSUT2    DD DSN=&PREFIX..TEMP(TSOOMVS),DISP=SHR
//SYSIN     DD DUMMY
//*
//*   CREATE A CLIST THAT CAN CUSTOMIZE THE SCRIPT & CONFIG FILES
//*
//USSEDIT EXEC PGM=IEBGENER
//SYSPRINT  DD SYSOUT=*
//SYSUT1    DD *,SYMBOLS=JCLONLY
PROC 3 PREFIX PRESMP STEPLIB
CONTROL LIST SYMLIST CONLIST NOFLUSH END(ENDD) ASIS
ERROR DO       /* keep going eg after a 'text not found' msg
   SET &POS = HANDLED       /* indicate error occurred
   SET &SAVECC = &LASTCC    /* save the error code
   RETURN                   /* ...to next stmt
```

```
ENDD
 /* this clist changes the commands to use &prefix from jcl
 /* a leading dot will have added to presmp so remove it first
IF &PRESMP = . THEN SET &PRESMP =
ELSE IF &SUBSTR(1:2,&PRESMP  ) = .. THEN -
   SET &PRESMP = &SUBSTR(2:&LENGTH(&STR(&PRESMP)),&PRESMP)
E '&PREFIX..TEMP(TSOOMVS)' TEXT ASIS
V ON
L
REN 10 10
CHANGE 1 9999 /--SMP--/&PRESMP/ ALL
CHANGE 1 9999 /--PREF--/&PREFIX/ ALL
IF &PRESMP = THEN CHANGE 1 9999 +DEFINE ALIAS+/*DEFINE ALIAS+ ALL
IF &PRESMP = THEN CHANGE 1 9999 +REPRO INDA+/*REPRO INDA+ ALL
L
UNNUM
L
END SAVE
E '&PREFIX..CNTL(DSCONF)' TEXT ASIS
V ON
L
REN 10 10
TOP
in # This is a copy of /etc/ldap/ds.conf modified for ldapx
in #
find 'adminDN' 1
change * 'adminDN' '#adminDN'
in adminDN "racfid=ldapadm,profiletype=user,cn=racf,o=site"
find 'adminPW' 1
change * 'adminPW' '#adminPW'
in #adminPW is not needed in here since adminDN is a RACF user
find 'commThreads' 1
change * 'commThreads' '#commThreads'
in commThreads 2
find 'listen ldap://:389 ' 1
change * ':389' ':389'
 /* #logfile //DD:LOGFILE
 /* logfile /var/ars/ldapibm/gldlog.output
 /* #schemaPath /var/ldap/schema
 /* schemaPath /var/ars/ldapibm/schema
find 'sizeLimit 500 ' 1
change * '500' '20000'
find '#srvStartUpError' 1
in # added ignore  so continues despite wlm init failure
in srvStartUpError ignore
 /* UNCOMMENT BELOW LINE FOR SSL_CERTIFICATE & RING_FILE
 /*find '#sslCertificate' 1
 /*in # see job in <ldapx>.cntl(ldapssl) for sample racf commands
 /*in sslCertificate LDAPSSLCERT
 /*find '#sslKeyRingFile' 1
 /*in # see job in <ldapx>.cntl(ldapssl) for sample racf commands
 /*in sslKeyRingFile LDAPSRV.RING
find '#plugin' 1
in plugin preOperation SDBM2TSO plugin_init -
"&USSDIR/logSDBM2TSO"
in plugin postOperation SDBM2TSO plugin_init -
"&USSDIR/logSDBM2TSO"
find '#database SDBM ' 1
in database SDBM GLDBSD31/GLDBSD64
find '#suffix ' 1
```

```
in suffix "cn=racf,o=site"
find '#enableResources' 1
in enableResources on
find '#sizeLimit 500 ' 1
in sizeLimit 10000
L
UNNUM
L
END SAVE
E '&PREFIX..CNTL(ENVVARS)' TEXT ASIS
V ON
L
REN 10 10
TOP
in # This is a copy of /etc/ldap/ds.envvars modified for ldapx
in #
find 'LANG=' 1
in LDAP_COMPAT_FLAGS=SDBM_MIXEDDN
L
UNNUM
L
END SAVE
E '&PREFIX..CNTL(DSSRV)' TEXT ASIS
V ON
L
REN 100 100
TOP
in //* This is a copy of GLD.SGLDSAMP(DSSRV) modified for LDAPx plugin
in //*
in //* To start TDS without debug trace messages:
in //*     S DSSRV
in //* To start TDS with debug trace messages from the plugin:
in //*     S DSSRV,PARMS='-D PLUGIN'
in //* To start TDS with more debug trace messages, from plugin & TDS:
in //*     S DSSRV,PARM='-D BER+PLUG+TRAC+THRE+SDBM'
in //* To alter debug tracing dynamically:
in //*     F DSSRV,DEBUG PLUG+SDBM+BER+PACK+FILT' etc
in //*     F DSSRV,DEBUG OFF'
in //* To stop the TDS LDAP server:
in //*     P DSSRV
in //*
in //* If need to remove the ldapx plugin:
in //*     Stop the server, edit ds.conf to comment out 2 lines that
in //*     have 'plugin' in column 1, restart the server.
in //* Or:
in //*     Stop the server, replace module SDBM2TSO in the STEPLIB
in //*     library with a renamed copy of IEFBR14, restart the server.
in //*     (this will call the renamed IEFBR14 as the plugin,
in //*     which will have the same effect as not calling the plugin)
in //*
find 'PGM=GLDSRVnn'
change * '//GO' '//*GO'
in //* Must use GLDSRV31 for LDAPX which does not need DB2 libraries.
in //* If some other LDAP service requires GLDSRV64 run 2 TDS servers.
in //GO        EXEC PGM=GLDSRV31,REGION=&&REGSIZE,TIME=1440,
find 'PARM=('
in //*   or make an alternative TDS config the default by editing this:
in //*          PARM=('/&&PARMS -f /etc/ldap/ds.conf >DD:DSOUT 2>&&1')
find '//*STEPLIB'
in //* If TDS cannot load the ldapx plugin, please check that the
```

```
in //* output of RACF command 'RLIST PROGRAM **' includes the dsname of
in //* the steplib library below, and the correct volume, and that this
in //* library is APF authorised, and that the plugin module (SDBM2TSO
in //* normally) is in the library.  See ...CNTL(LDAPRACF) for details.
in //* If the plugin was moved to a lnklst library making the steplib
in //* irrelevant, the above also applies to the lnklst library.
in //STEPLIB    DD DSN=&STEPLIB,DISP=SHR
in //* The next line is referred to by the IBM comments about SDSNLOD2
in //* and since ldapx uses GLDSRV31 can be left commented out.
find '//*---'
in //* The following contains LDAPX configuration keywords
in //* as documented in the install doc.  Please review & customize.
in //* If none are supplied then LDAPX will run in basic mode,
in //* meaning without support for TSO commands or dataset profiles.
in //* A valid licensecode keyword is required to avoid basic mode.
in //*-------------------------------------------------------------------*
in //LDAPXCFG   DD DSN=&PREFIX..CNTL(LDAPXCFG),DISP=SHR
in //*-------------------------------------------------------------------*
in //* The following is for alternate TDS config, not for LDAPX config.
bottom
in //*SYSPRINT  DD SYSOUT=X   no longer needed...
in //*INDD1     DD DISP=SHR,DSN=SYS1.RACFDS                for irrdbu00
in //*OUTDD     DD DISP=SHR,DSN=SYS2.RACF.UNLOAD.ZDRES1.B  for irrdbu00
L
UNNUM
L
END SAVE
EXIT
//SYSUT2    DD DSN=&PREFIX..TEMP(TSOEDIT4),DISP=SHR
//SYSIN     DD DUMMY
//*
//*   RUN THE EDIT OF THE INSTALL SCRIPT FOR OMVS
//*
//EDIT4   EXEC PGM=IKJEFT01,DYNAMNBR=40,   EDIT ...TEMP(TSOOMVS)
//             PARM='%TSOEDIT4 &PREFIX .&PRESMP &STEPLIB'
//SYSPROC   DD DSN=&PREFIX..TEMP,DISP=SHR
//SYSTSPRT  DD TERM=TS,SYSOUT=*,HOLD=YES   /* TERMINAL OUTPUT
//SYSTSIN   DD DUMMY
//*================== 2nd stop here before continuing =================
//
//*========= restart here after commenting out the null above =========
//PART3   EXEC PGM=IEFBR14         easier to remember restart label
//*
//*   COPY THE EDITED SCRIPT TO USS READY FOR RUNNING
//*   WARNING: only run these steps if ok to overwrite TDS config
//*   otherwise manually edit ds.conf & ds.envvars and use compare
//*
//COPYIN  EXEC PGM=IKJEFT01
//SYSTSPRT  DD SYSOUT=*
//OMVSOUT   DD PATH='&USSDIR/ldapx_inst1',
//             PATHOPTS=(OWRONLY,OCREAT,OTRUNC),
//             PATHMODE=SIRWXU
//MVSIN     DD DSN=&PREFIX..TEMP(TSOOMVS),DISP=SHR  from step tsoomvs
//SYSTSIN   DD *
OCOPY INDD(MVSIN) OUTDD(OMVSOUT) TEXT CONVERT(YES) PATHOPTS(USE)
//*
//*   RUN USS SCRIPT TO COPY CUSTOMIZED OMVS FILES TO LDAPX DIRECTORY
//*
//OMVS    EXEC PGM=BPXBATCH,REGION=40M
//* PARM='SH make -f mjs.mk'
```

```
//* PARM=('SH command ldapmjs -h 127.0.0.1 -v -b "o=axent',
//*        'c=token" tser=1*')
//SYSPRINT  DD TERM=TS,SYSOUT=*,HOLD=YES
//SYSTERM   DD TERM=TS,SYSOUT=*,HOLD=YES
//SYSTSPRT  DD TERM=TS,SYSOUT=*,HOLD=YES   /* TERMINAL OUTPUT
//IEFRDER   DD TERM=TS,SYSOUT=*,HOLD=YES
//STDIN     DD PATH='&USSDIR/ldapx_inst1',
//          PATHOPTS=(ORDONLY)
//STDOUT    DD PATH='&USSDIR/ldapx_inst1.log',
//             PATHOPTS=(OWRONLY,OCREAT,OTRUNC),
//             PATHMODE=SIRWXU
//*STDERR    DD PATH='/mjsout.txt',   /* no need - default same
//*             PATHOPTS=(OWRONLY,OCREAT,OTRUNC),
//*             PATHMODE=SIRWXU
//STDENVX   DD *
PATH=/usr/lpp/internet/samples/API
//*
//*DEL      EXEC DEL,D='IBMUSER.LDAPOUT1.TXT'
//*
//COPYLOG1 EXEC PGM=IKJEFT01
//TXTOUT    DD PATH='&USSDIR/ldapx_inst1.log',
//             PATHOPTS=(ORDONLY)
//SPOOL     DD SYSOUT=*,
//             DCB=(RECFM=VB,LRECL=256,BLKSIZE=24000)
//SYSTSPRT  DD SYSOUT=*
//SYSTSIN   DD *
OCOPY INDD(TXTOUT) OUTDD(SPOOL) TEXT CONVERT(YES) PATHOPTS(USE)
//* OCOPY INDD(TSTERR) OUTDD(OUTMVS) TEXT CONVERT(YES) PATHOPTS(USE)
//*
//*GENOUT  EXEC PGM=IEBGENER
//*SYSPRINT  DD DUMMY
//*SYSUT1    DD DSN=IBMUSER.LDAPOUT1.TXT,DISP=SHR
//*SYSUT2    DD SYSOUT=*
//*SYSIN     DD DUMMY
//*
//*
//*
//*   COPY THE 'INSTALL to LIVE DIR'  SCRIPT TO USS READY FOR RUNNING
//*
//LDINST   EXEC PGM=IKJEFT01
//SYSTSPRT  DD SYSOUT=*
//OMVSOUT   DD PATH='&USSDIR/ldapx_inst2',
//             PATHOPTS=(OWRONLY,OCREAT,OTRUNC),
//             PATHMODE=SIRWXU
//MVSINY    DD DSN=&PREFIX..TEMP(TSOOMVS),DISP=SHR
//MVSIN     DD *,SYMBOLS=JCLONLY
# turn on listing each command
set -x
# can no longer mkdir from here since jcl now uses the directory!
#cd /usr
#mkdir oneiden
#
ussdir='&USSDIR'
pds='&PREFIX'
cd $ussdir
# take backup of current TDS config (if any exist)
cat   /etc/ldap/ds.conf
cp -T /etc/ldap/ds.conf    $ussdir/ds.conf-original
cat   /etc/ldap/ds.envvars
cp -T /etc/ldap/ds.envvars $ussdir/ds.envvars-original
```

```
cat   /bin/tsocmd
cp -T /bin/tsocmd          $ussdir/tsocmd-original
cp -T /bin/tsocmd "//'$pds.REPRO(TSOCMDO)'"
cp -T "//'$pds.REPRO(SCRIPTSO)'" $ussdir/scriptso
cp -T "//'$pds.REPRO(SCRIPTSE)'" $ussdir/scriptsoecho
chmod 775 scriptso
ls -alHi
#
cd /etc
mkdir ldap
cd /etc/ldap
#####################################################################
# be sure you want to create or overwrite the 'live' TDS config     #
# before running the next 2 copy commands                           #
# (as a safeguard backups should have been taken by commands above) #
#####################################################################
cp -T "//'$pds.CNTL(dsconf)'" /etc/ldap/ds.conf
cp -T "//'$pds.CNTL(envvars)'" /etc/ldap/ds.envvars
#ls -alHi
#
#  no need to copy tsocmd since IBM supply it now...
#cp -T "//'$pds.REPRO(TSOCMD)'" /bin/tsocmdx (ibm std now)
//SYSTSIN   DD *
OCOPY INDD(MVSIN) OUTDD(OMVSOUT) TEXT CONVERT(YES) PATHOPTS(USE)
//*
//*   RUN USS SCRIPT TO COPY OMVS COMPONENTS TO RUNTIME DIRECTORIES
//*
//OMVS    EXEC PGM=BPXBATCH,REGION=40M
//* PARM='SH make -f mjs.mk'
//* PARM=('SH command ldapmjs -h 127.0.0.1 -v -b "o=axent',
//*        'c=token" tser=1*')
//SYSPRINT  DD TERM=TS,SYSOUT=*,HOLD=YES
//SYSTERM   DD TERM=TS,SYSOUT=*,HOLD=YES
//SYSTSPRT  DD TERM=TS,SYSOUT=*,HOLD=YES   /* TERMINAL OUTPUT
//IEFRDER   DD TERM=TS,SYSOUT=*,HOLD=YES
//STDIN     DD PATH='&USSDIR/ldapx_inst2',
//             PATHOPTS=(ORDONLY)
//STDOUT    DD PATH='&USSDIR/ldapx_inst2.log',
//             PATHOPTS=(OWRONLY,OCREAT,OTRUNC),
//             PATHMODE=SIRWXU
//*
//COPYLOG2 EXEC PGM=IKJEFT01
//TXTOUT    DD PATH='&USSDIR/ldapx_inst2.log',
//             PATHOPTS=(ORDONLY)
//SPOOL     DD SYSOUT=*,
//             DCB=(RECFM=VB,LRECL=256,BLKSIZE=24000)
//SYSTSPRT  DD SYSOUT=*
//SYSTSIN   DD *
OCOPY INDD(TXTOUT) OUTDD(SPOOL) TEXT CONVERT(YES) PATHOPTS(USE)
//*
//*   COPY JCL TO PROCLIB FOR RUNNING IBM TIVOLI DIRECTORY SERVER
//*   (may need to edit after if using diff ds.conf envvars or schema)
//*
//PROCLIB EXEC PGM=IEBCOPY
//SYSPRINT  DD SYSOUT=*
//SYSUT1    DD DISP=SHR,DSN=&PREFIX..CNTL
//SYSUT2    DD DISP=SHR,DSN=&PROCLIB
//SYSUT3    DD UNIT=VIO,SPACE=(CYL,(5,5))
//SYSUT4    DD UNIT=VIO,SPACE=(CYL,(5,5))
//SYSIN     DD *
```

```
* COPY INDD=SYSUT1,OUTDD=SYSUT2           /* copy noreplace
*   or
* COPY INDD=((SYSUT1,R)),OUTDD=SYSUT2     /* copy replace
*   then
* S M=(DSSRV)   /* stc jcl for tds with ldapx extension plugin
*   or
* S M=((DSSRV,DSSRV22))  /* copy giving new name for jcl proc
*
 COPY INDD=SYSUT1,OUTDD=SYSUT2
 S M=(DSSRV)   /* stc jcl for tds with ldapx extension plugin
//*
//*   COPY LDAPX PLUGIN TO STEPLIB OF IBM TIVOLI DIRECTORY SERVER
//*
//STEPLIB EXEC PGM=IEBCOPY
//SYSPRINT  DD SYSOUT=*
//SYSUT1    DD DISP=SHR,DSN=&PREFIX..LOAD
//SYSUT2    DD DISP=SHR,DSN=&STEPLIB
//SYSUT3    DD UNIT=VIO,SPACE=(CYL,(5,5))
//SYSUT4    DD UNIT=VIO,SPACE=(CYL,(5,5))
//SYSIN     DD *
* COPYMOD INDD=SYSUT1,OUTDD=SYSUT2        /* copy noreplace
*   or
* COPYMOD INDD=((SYSUT1,R)),OUTDD=SYSUT2  /* copy replace
* S M=(SDBM2TSO)             /* ldapx extension plugin for ibm tds
*   or   (alternative name must be coded on plugin stmts in ds.conf)
* S M=((SDBM2TSO,SDBM2V62))  /* copy giving plugin an alternative name
*
 COPYMOD INDD=SYSUT1,OUTDD=SYSUT2
 S M=(SDBM2TSO)   /* ldapx extension plugin for ibm tds (std name)
//
```